# Terraform Project

백우진 서승덕 양혜원 한준영

# 목차


01 Terraform 정의
- Terraform - IaC
- 프로비저닝 - 장점

02 환경 구축
- Terraform - AWS CLI
- VSCode

03 Terraform 기본 개념
- provider - resource
- variable - output
- terraform.tfstate 파일

04 Terraform 작동 원리
- init - validate - plan
- apply - destroy

05 TF 파일 분석
- 구조 및 구성도 - Code

06 GitHub, Terraform Cloud, AWS 연동
- GitHub Repository 만들기
- Terraform Cloud 초기 구성 & Git hub 연동
- Terraform Cloud & AWS 연동

07 Route53 도메인 등록
- 호스트 영역 생성
- 도메인 등록업체 NS 레코드 입력

# 1. Terraform 정의

## 1-1. Terraform

- HashiCorp에서 개발한 오픈소스 인프라 관리 도구로,인프라를 코드(IaC) 형태로 작성해 자동화된 관리 가능
- 선언적 구문(HCL/JSON)을 사용하여 사용자가 원하는 인프라를 정의하면 Terraform이 자동으로 구축 처리
- AWS, Azure, GCP 등 멀티 클라우드 및 온프레미스 환경의 자원 프로비저닝과 관리 지원

## 1-2. IaC

- IaC는 "인프라를 코드로 관리한다 " 는 개념으로, 서버•네트워크•DB 설정을 코드화하여 자동화
- 선언적(Declarative) 방식과 명령적(Imperative) 방식을 통해 인프라를 정의하고 운영
- Terraform 외에도 CloudFormation, Ansible, Chef, Kubernetes Manifests 등 다양한 도구 존재

## 1-3. 프로비저닝

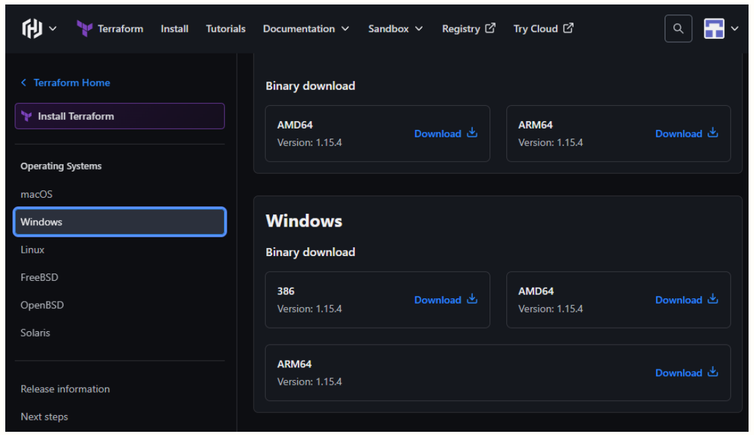
- IT 시스템에서 서버, 네트워크, 데이터베이스 등 필요한 자원을 준비하고 설정하는 과정
- 클라우드에서 서버 생성, 데이터베이스 설정, 네트워크 할당 등이 포함
- 자동화 도구를 사용하면 구축 시간을 단축하고 오류를 줄이며, 확장성 있는 인프라 관리가 가능

## 1-4. 장점

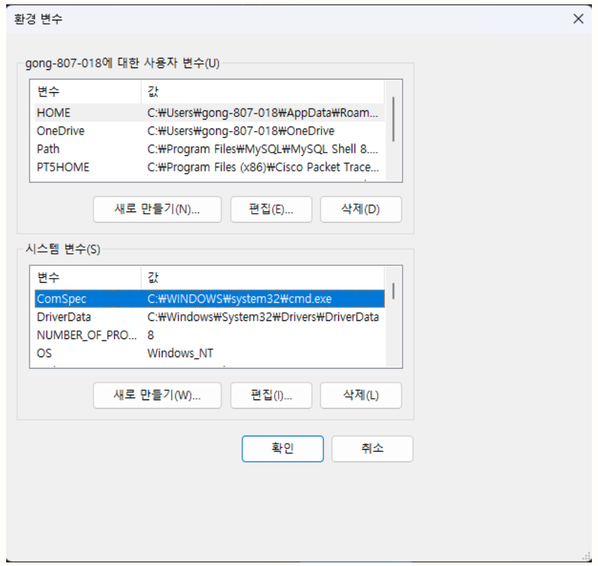
- 자동화: Terraform으로 코드 기반 인프라 관리를 통해 반복 작업을 줄이고, 빠르고 정확하게 리소스를 배포
- 일관성: 동일한 코드를 사용해 개발 , 태스트, 운영 환경 간 차이를 없애고 예측 하능한 인프라 구성이 가능
- 효율성 및 버전 관리: Git으로 코드 변경  이력을 관리하고, CI/CD와 연동해 안정적으로 배포할 수 있으며, 복잡한 인프라도 빠르게 운영

# 2. 환경 구축

## 2-1. Terraform 설치

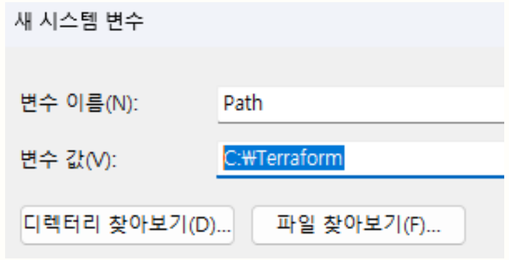



Windows AMD64 다운로드




시스템 환경 변수 편집 -- 고급 -- 환경 변수

시스템 변수의 Path에 Terraform 경로 추가




새로 만들기 -- 변수이름 = Path, 변수 값 = C:₩Terraform 입력

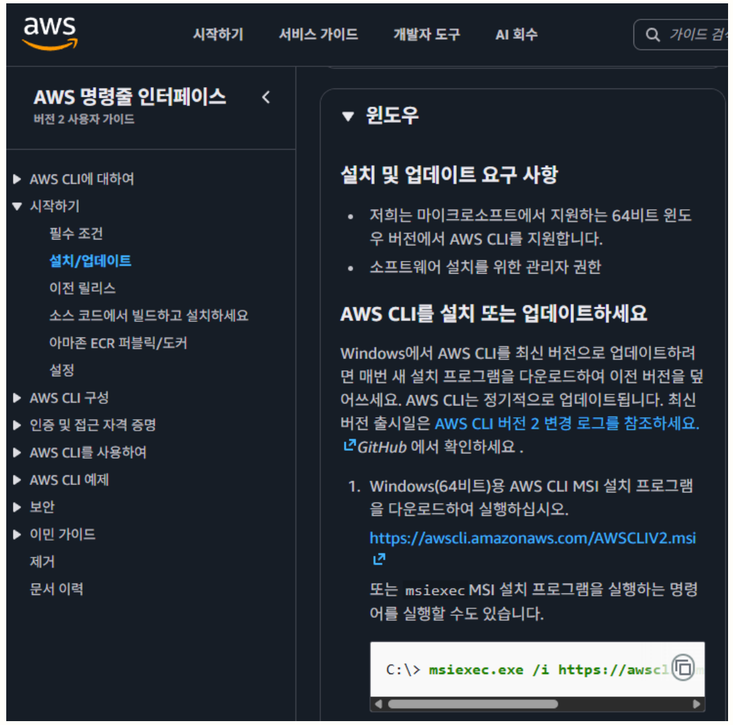
변수 이름을 Path로 주지 않으면 인식 못함

```
C:\Users\gong-807-018>terraform version
Terraform v1.15.3
on windows_amd64
+ provider registry.terraform.io/hashicorp/aws v5.100.0
+ provider registry.terraform.io/hashicorp/local v2.9.0
+ provider registry.terraform.io/hashicorp/tls v4.3.0
```

Terraform 설치 확인

# 2. 환경 구축

## 2-2. AWS CLI 연동




https://awscli.amazonaws.com/AWSCLI2.msi
다운로드 후 실행

```
C:\Users\gong-807-018>terraform version
Terraform v1.15.3
on windows_amd64
+ provider registry.terraform.io/hashicorp/aws v5.100.0
+ provider registry.terraform.io/hashicorp/local v2.9.0
+ provider registry.terraform.io/hashicorp/tls v4.3.0
```

CMD에 aws --version 입력




AWS -- IAM -- 엑세스 키 관리 -- 액세스 키 만들기

# 2. 환경 구축

## 2-2. AWS CLI 연동

```
C:\Users\gong-807-018>aws configure
AWS Access Key ID [****************T4UL]:
AWS Secret Access Key [****************URim]:
Default region name [json]:
Default output format [None]:
```

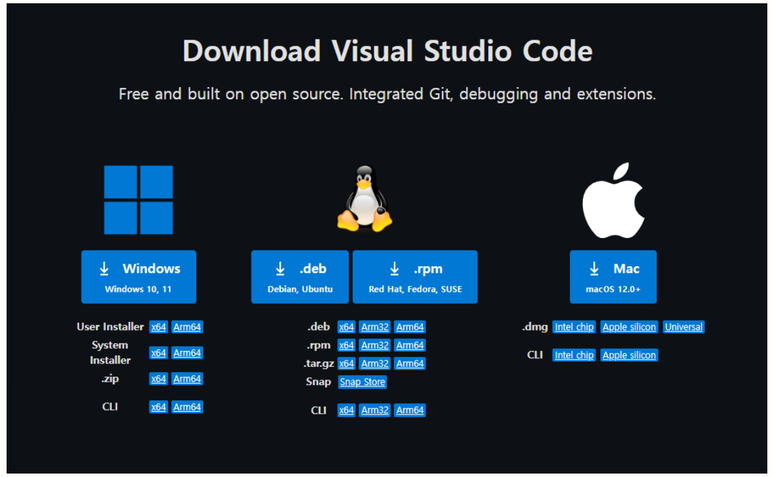
CMD에 aws configure 입력
Access Key ID, Secrest Access Key,
Default region 입력

```
C:\Users\gong-807-018>aws configure list
NAME       : VALUE                   : TYPE                 : LOCATION
profile    : <not set>               : None                 : None
access_key : ****************T4UL    : shared-credentials-file :
secret_key : ****************URim    : shared-credentials-file :
region     : json                    : config-file          : ~/.aws/config
```

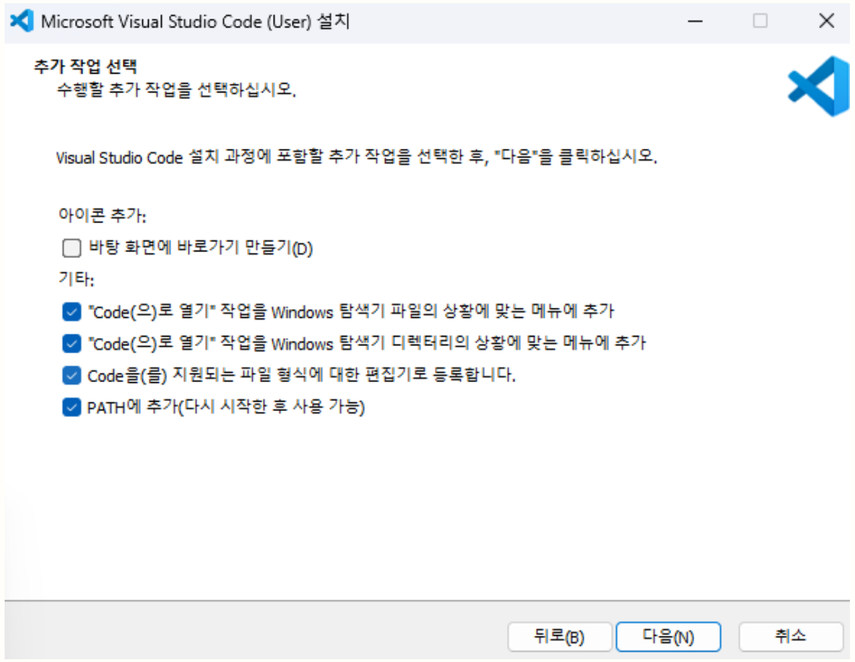
CMD에 aws configure list 입력
AWS 로그인 확인

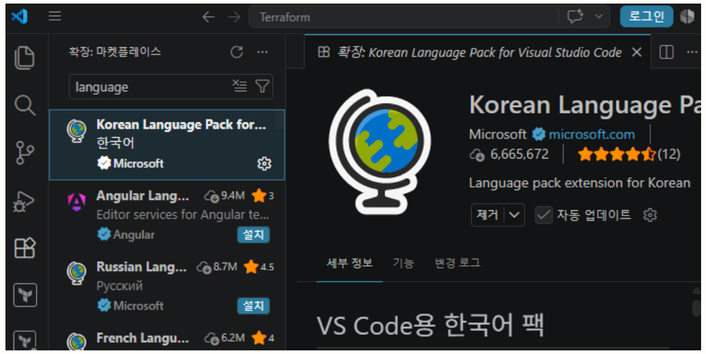
# 2. 환경 구축

## 2-3. VSCode 설치




Windows System Installer x64 설치



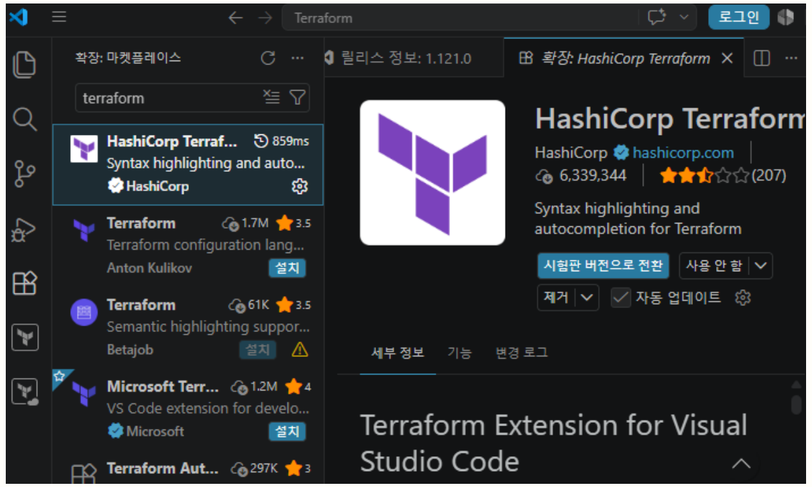

Language 검색 -- 한국어 선택

Code을(를) 지원되는 파일 형식에 대한 편집기로 등록합니다.

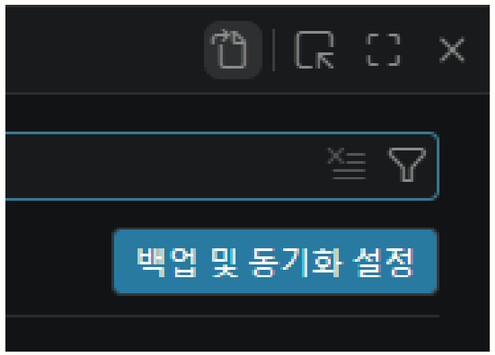
PATH에 추가(다시 시작한 후 사용 가능)

# 2. 환경 구축

## 2-3. VSCode 설치




HashiCrop Terraform 설치




settings.json C:\Users\gong-807-018\AppData\Roaming\Code\User

C: > Users > gong-807-018 > AppData > Roaming > Code > User > {} settings.json >

```
{
"editor.formatOnSave": true,
"editor.formatOnSaveMode": "file",
"[terraform]": {
"editor.defaultFormatter": "hashicorp.terraform"
},
"[terraform-vars]": {
"editor.formatOnSave": true,
"editor.defaultFormatter": "hashicorp.terraform",
"editor.formatOnSaveMode": "file"
},
"editor.language.brackets": []
}
```

아래 JSON 내용 추가 후 저장(Ctrl+S)

Terraform 파일을 자동으로 포맷팅하도록 구성하는 설정

Settings -- 오른쪽 상단 파일 아이콘(Open Settings HSON) 클릭

# 3. Terraform 기본 개념

## 3-1. provider

Terraform이 인프라를 생성할 때 사용할 클라우드 플랫폼이나 서비스를 정의

각 클라우드 플랫폼에 맞는 provider를 지정해야 인프라를 생성할 수 있음

프로바이더는 Terraform 핵심 소프트웨어와는 독립적으로 출시되는 플러그인이며, 자체적인 버전 관리를 가집니다.

# 3. Terraform 기본 개념

## 3-2. resource

실제로 생성할 인프라 자원을 정의 EC2 인스턴스, VPC, EDS 등 리소스를 프로비저닝

리소스 블록은 테라폼에서 관리한 인프라 구성 요소를 정의하는 기본적인 빌딩 블록

사용법: resource 키워드를 사용해 정의, 각 블록은 리소스 유형과 구성 내에서 이를 참조할 이름을 지정

# 3. Terraform 기본 개념

## 3-3. variable

인프라 구성 코드를 유연하고 재사용 가능하게 만드는데 필수적인 요소 마치 프로그래밍 언어의 변수처럼, 하드코딩 된 값을 직접 사용하는 대신 변수를 통해 값을 전달받아 사용 가능

코드에서 값이 변경될 수 있는 파라미터를 정의 변수는 사용자로부터 입력 받거나, 다른 모듈에서 값을 전달받을 때 사용

variable을 사용하면 코드의 재사용성과 유연성을 높일 수 있음

# 3. Terraform 기본 개념

## 3-4. output

Terraform 구성에서 생성된 값을 사용자나 다른 시스템에서 사용할 수 있도록 출력하는 기능
배포 후 리소스 정보(예: IP 주소, DNS 이름 등)를 사용자에게 출력

인프라 구성 코드를 유연하고 재사용 가능하게 만드는 데 필수적인 요소 마치 프로그래밍 언어의 변수처럼, 하드코딩 된 값을 직접 사용하는 대신 변수를 통해 값을 전달받아 사용 가능

# 3. Terraform 기본 개념

## 3-5. terraform.tfstae 파일

Terraform이 관리하는 인프라의 현재 상태를 저장하는 파일 Terraform이 인프라 리소스의 상태를 추적하고 관리 명령을 실행할 때 참조

Terraform이 배포한 리소스(VPC, EC2 인스턴스 등)의 현재 상태 정보를 JSON 형식으로 저장 version, resources, outputs, modules 등의 정보가 저장되어 있음

팀 협업 시 Terraform Backend(S3, Azure Blob Storage 등)를 설정하여 상태 파일을 원격에서 관리

# 4. Terraform 작동원리

## 4-1. init

Terraform 작업 디렉토리를 초기화하고, 구성 파일에 정의된 Provider 플러그인을 다운로드 작업 디렉토리를 테라폼이 사용할 수 있는 환경으로 설정 .terraform 디렉토리가 생성됨
사용자의 테라폼 코드가 들어갈 작업 디렉터리를 초기화
모듈과 플러그인 같은 보조적인 구성요소를 다운로드
테라폼 상태 파일을 저장할 백 엔드를 구성

## 4-2. validate

구성 파일의 문법과 논리적 유료성을 검사하여 오류를 미리 확인하는 명령어 Terraform plan에서도 구문 오류를 확인하기에 생략 가능
로컬에서만 검사를 수행하며 원격 서비스나 상태와 상호작용하지 않음
CI/CD 파이프라인에서는 유용함, 파이프라인 단계에서 validate를 먼저 실행해 코드 오류를 빠르게 확인하고, 코드 구문이 올바른 경우에만 plan을 실행하도록 설정해서 불필요한 작업 및 리소스를 줄일 수 있음

## 4-3. plan

현재 상태와 코드에서 정의된 원하는 상태 간의 차이를 계산하여, 적용할 변경 사항을 미리 보여줌, 변경 사항이 실제 인프라에 적용되기 전에 검토할 수 있도록 하는 기능
수행할 작업을 시뮬레이션하여  예측 결과를 보여줌
Kubernetes 파드 생성 명령어중에서 –dry-run=client와 유사
상태 파일(terraform.tfstate) 및 클라우드 환경과 상호작용 사용자는 변경 사항을 적용하기 전에 어떤 작업이 수행될지 미리 검토 가능

## 4-4. apply

Terraform plan에서 계산된 변경 사항을 실제로 인프라에 적용
적용 후, 변경된 상태를 상태 파일(terraform.tfstate)에 저장
옵션 -‘auto-approve’: 승인 없이 실행, ‘replace=<resource>’: 특정 리소스 강제 재생성
일부 리소스 생성/변경이 실패하면, 그 시점에서 멈추고 상태는 반영된 부분까지만 저장 , 자동으로 롤백하지 않기 때문에 수동 정리 필요

# 4. Terraform 작동원리

## 4-5. destroy

Terraform  구성 파일에 정의된 모든 리소스를 삭제
상태 파일(terraform.tfstate)도 업데이트 됨

단, destroy는 apply로 생성한 인프라 리소스의 설정 사항 변동이 없을 때만 가능
(임의 변경 후 destroy 실행 시, 지워지지 않고 에러 발생)

terraform destroy –auto-approve : 확인 없이 바로 삭제
terraform destroy –target=aws_instance.my_ec2 : 특정 리소스만 삭제

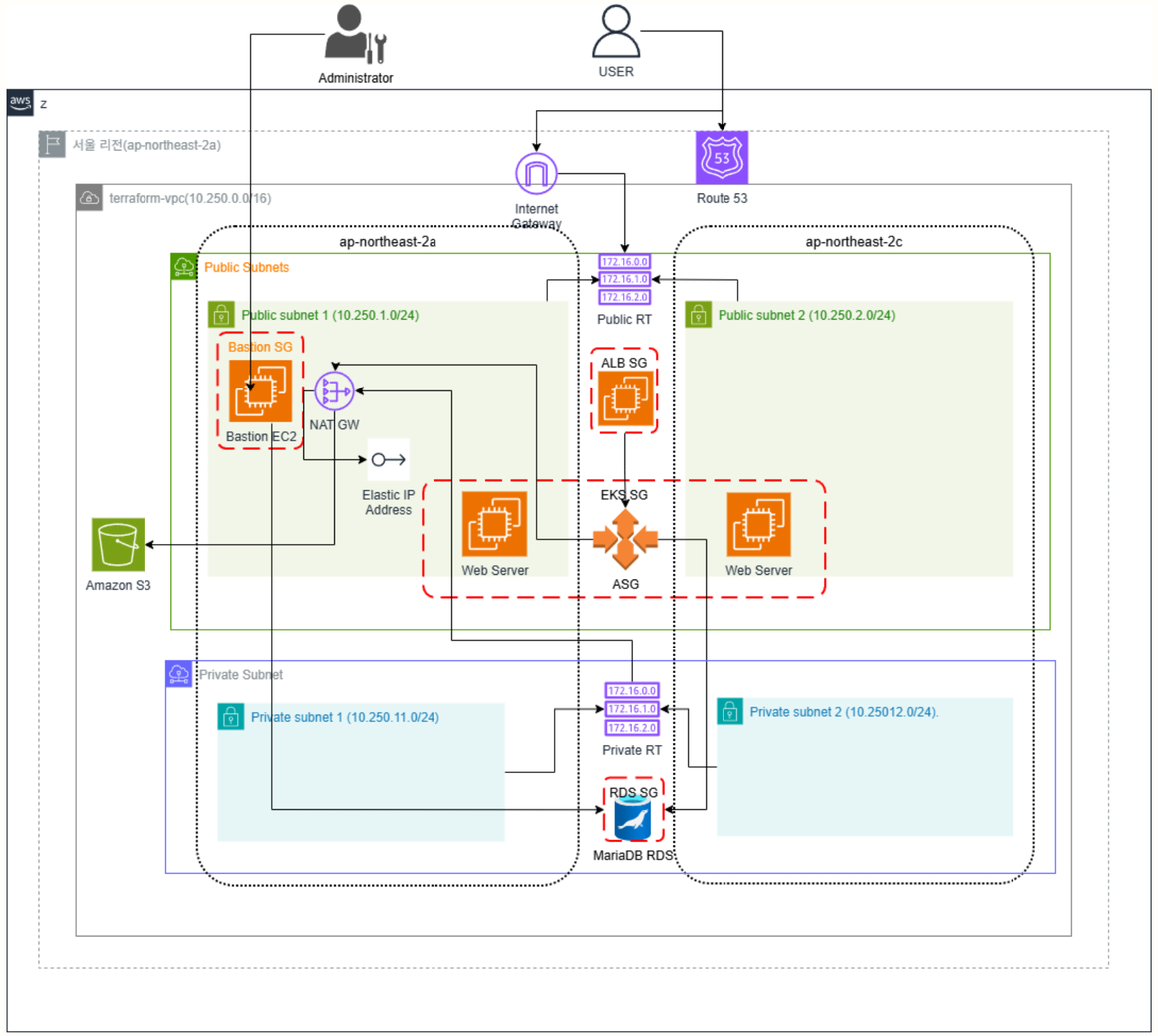
# 5. TF 파일 분석

## 5-1. 구조 및 구성도

<구조>

Terraform-Class/

- 01. terraform provider.tf
- 02. VPC.tf
- 03. Subnet.tf
- 04. IGW.tf
- 05. NGW.tf
- 06. RT.tf
- 07. Security-Group.tf
- 08. Key_pair.tf
- 09. EC2.tf
- 10. RDS.tf
- 11. S3.tf
- 12. acm.tf
- 13. route53 .tf
- 14. ALB.tf
- 15. AutoScailing.tf



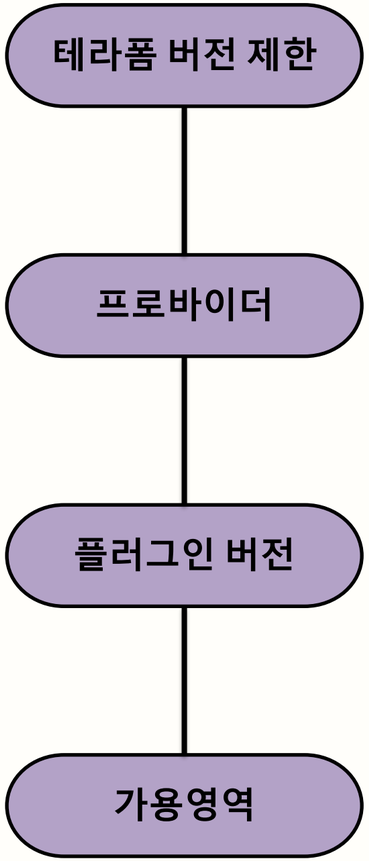
# 1. Terraform provider

```
terraform {
  required_version = ">= 1.0.0, <2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-northeast-2"
}
```

# 5. TF 파일 분석




1.0.0 이상 2.0.0 미만

aws

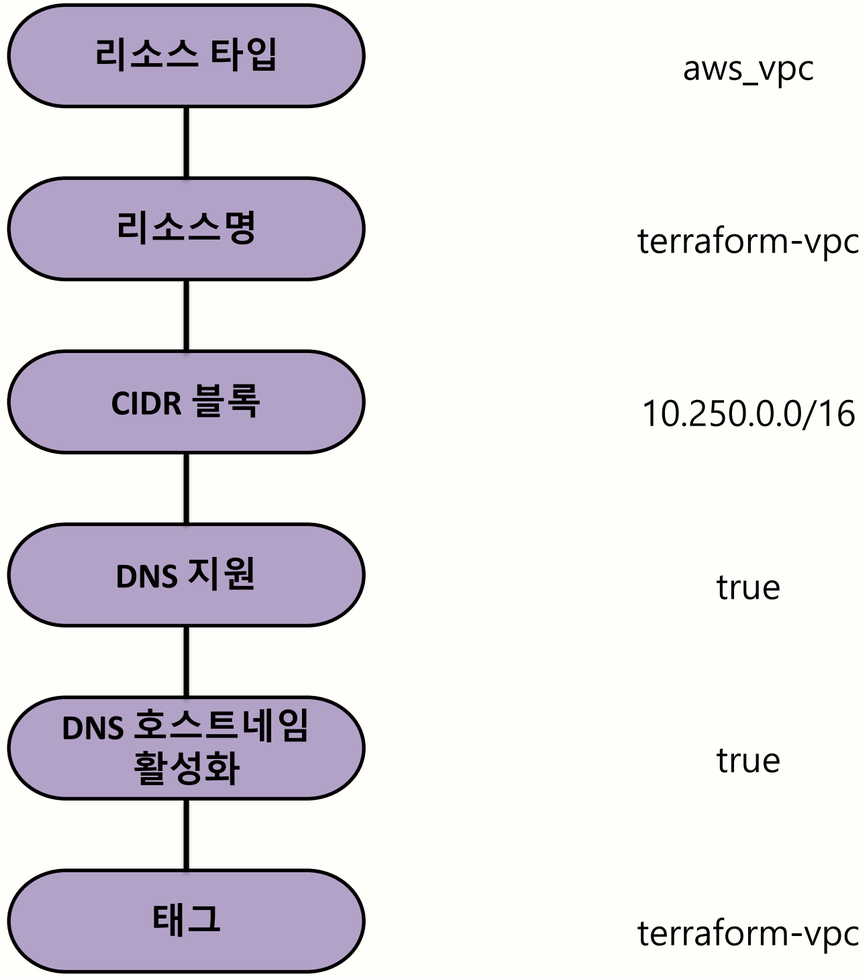
5.0 이상 6.0 미만

region= = “ap-northeast-2”
#Asia Pacific (seoul) region

# 2. VPC

# 5. TF 파일 분석

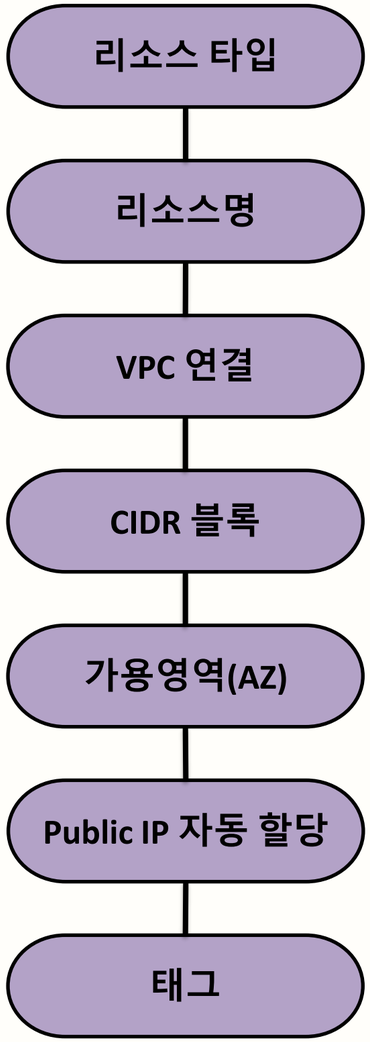
```
resource "aws_vpc" "terraform-vpc" {
  cidr_block           = "10.250.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = {
    "Name" = "terraform-vpc"
  }
}
```



# 3. Subnet (Public 2a&2c 동일)

# 5. TF 파일 분석

```
resource "aws_subnet" "terraform-pub-subnet-2a" {
  vpc_id      = aws_vpc.terraform-vpc.id
  cidr_block      = "10.250.1.0/24"
  availability_zone   = "ap-northeast-2a"
  map_public_ip_on_launch = "true"
  tags = {
    "Name"                                          =
"terraform-pub-subnet-2a"
    "kubernetes.io/role/elb"                              = "1"
    "kubernetes.io/cluster/terraform-eks-cluster" =
"shared"
  }
}
...
```




aws_subnet

terraform-pub-subnet-2a
terraform-pub-subnet-2c

terraform-vpc

10.250.1.0/24 -> 서브넷 1 IP 범위
10.250.2.0/24 -> 서브넷 2 IP 범위

ap-northeast-2a
ap-northeast-2c

true -> 생성되는 인스턴스가
자동으로 공인 IP 할당

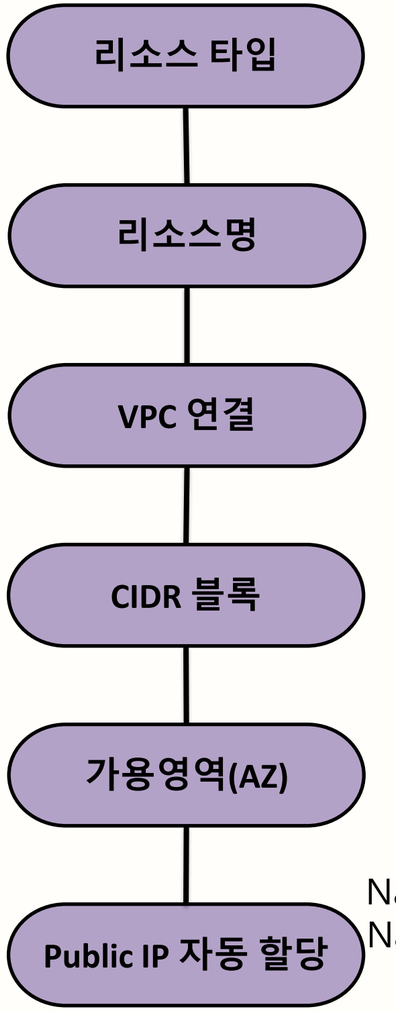
Name = terraform-pub-subnet-2a -> 관리용 식별 태그
Name = terraform-pub-subnet-2c -> 관리용 식별 태그
kubernetes.io/role/elb = 1 -> EKS ALB 연동용
kubrnetes.io/cluster/terraform-eks-cluster = shared

# 3. Subnet (Private 2a&2c 동일)

```
resource "aws_subnet" "terraform-pri-subnet-2a" {
  vpc_id                  = aws_vpc.terraform-vpc.id
  cidr_block              = "10.250.11.0/24"
  availability_zone = "ap-northeast-2a"
  tags = {
    "Name"                                          =
"terraform-pri-subnet-2a"
    "kubernetes.io/role/internal-elb"               = "1"
    "kubernetes.io/cluster/terraform-eks-cluster" =
"shared"
  }
}

...
```

# 5. TF 파일 분석




aws_subnet

terraform-pri-subnet-2a
terraform-pri-subnet-2c

terraform-vpc

10.250.11.0/24 -> 서브넷 1 IP 범위
10.250.12.0/24 -> 서브넷 2 IP 범위

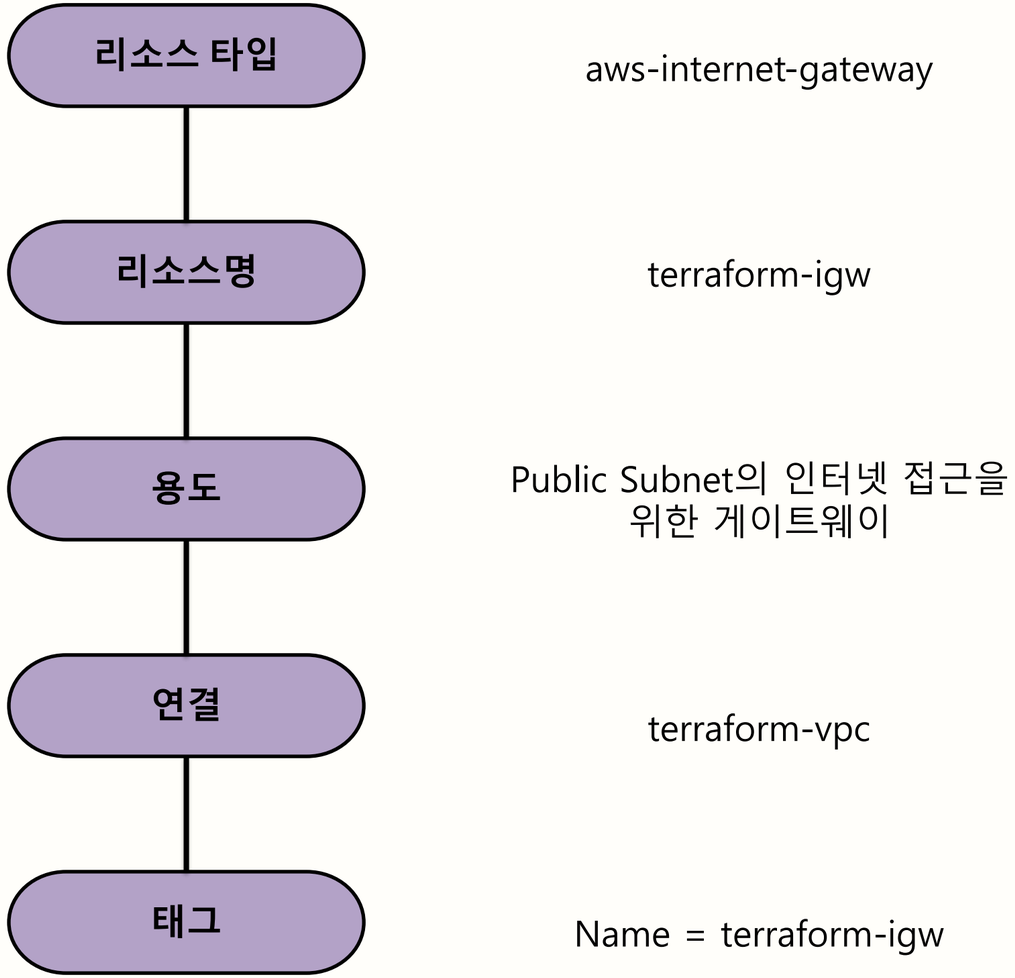
ap-northeast-2a
ap-northeast-2c

Name = terraform-pub-subnet-2a -> 관리용 식별 태그
Name = terraform-pub-subnet-2c -> 관리용 식별 태그
kubernetes.io/role/elb = 1 -> EKS ALB 연동용
kubrnetes.io/cluster/terraform-eks-cluster = shared

# 4. IGW

```
resource "aws_internet_gateway" "terraform-igw" {
  vpc_id = aws_vpc.terraform-vpc.id
  tags = {
    "Name" = "terraform-igw"
  }
}
```

# 5. TF 파일 분석

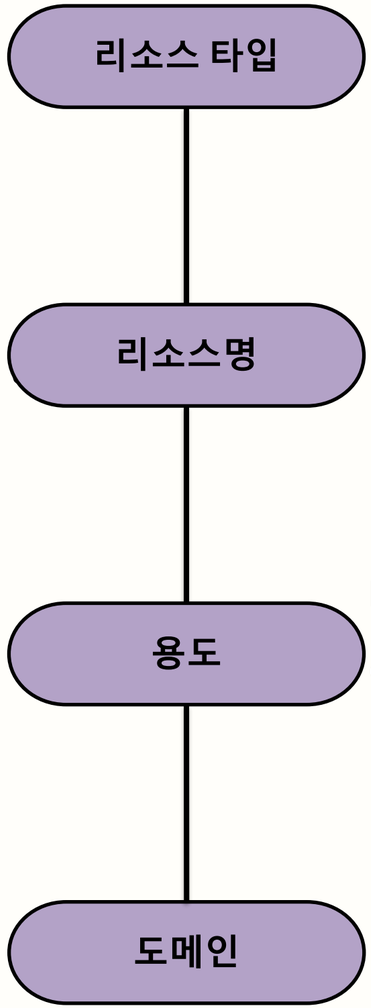


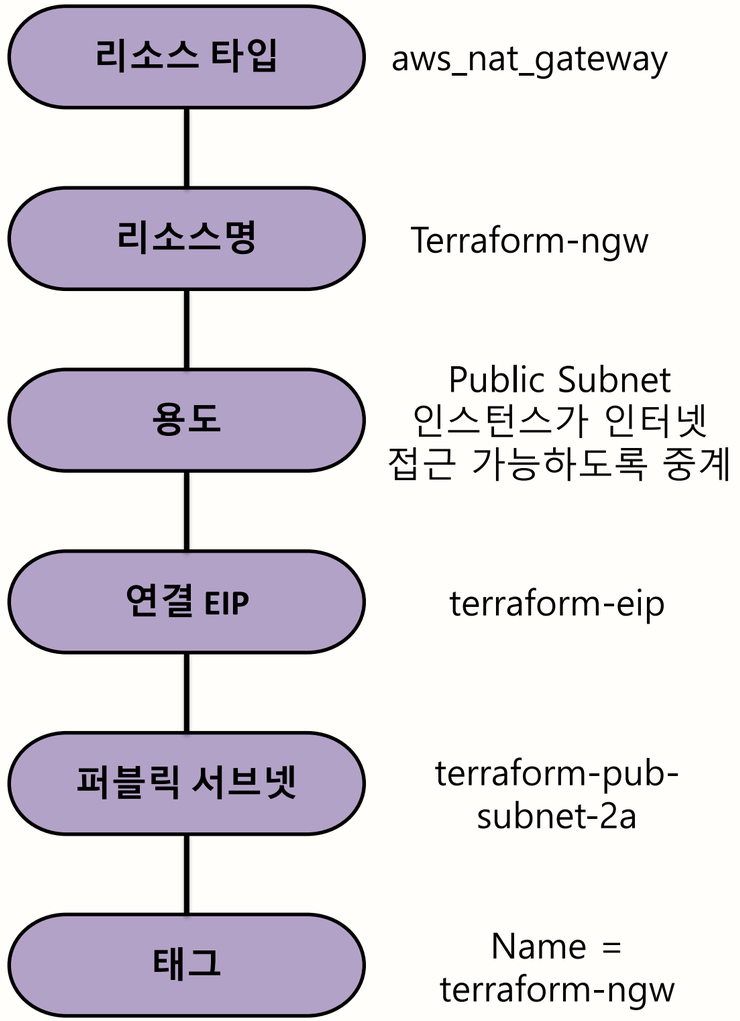
# 5. NGW

# 5. TF 파일 분석

```
# 탄력적 IP
resource "aws_eip" "terraform-eip" {
  domain = "vpc"
}

# NAT 게이트웨이
resource "aws_nat_gateway" "terraform-ngw" {
  allocation_id = aws_eip.terraform-eip.id
  subnet_id     = aws_subnet.terraform-pub-
subnet-2a.id
  tags = {
    "Name" = "terraform-ngw"
  }
}
```



# 6. Routing Table



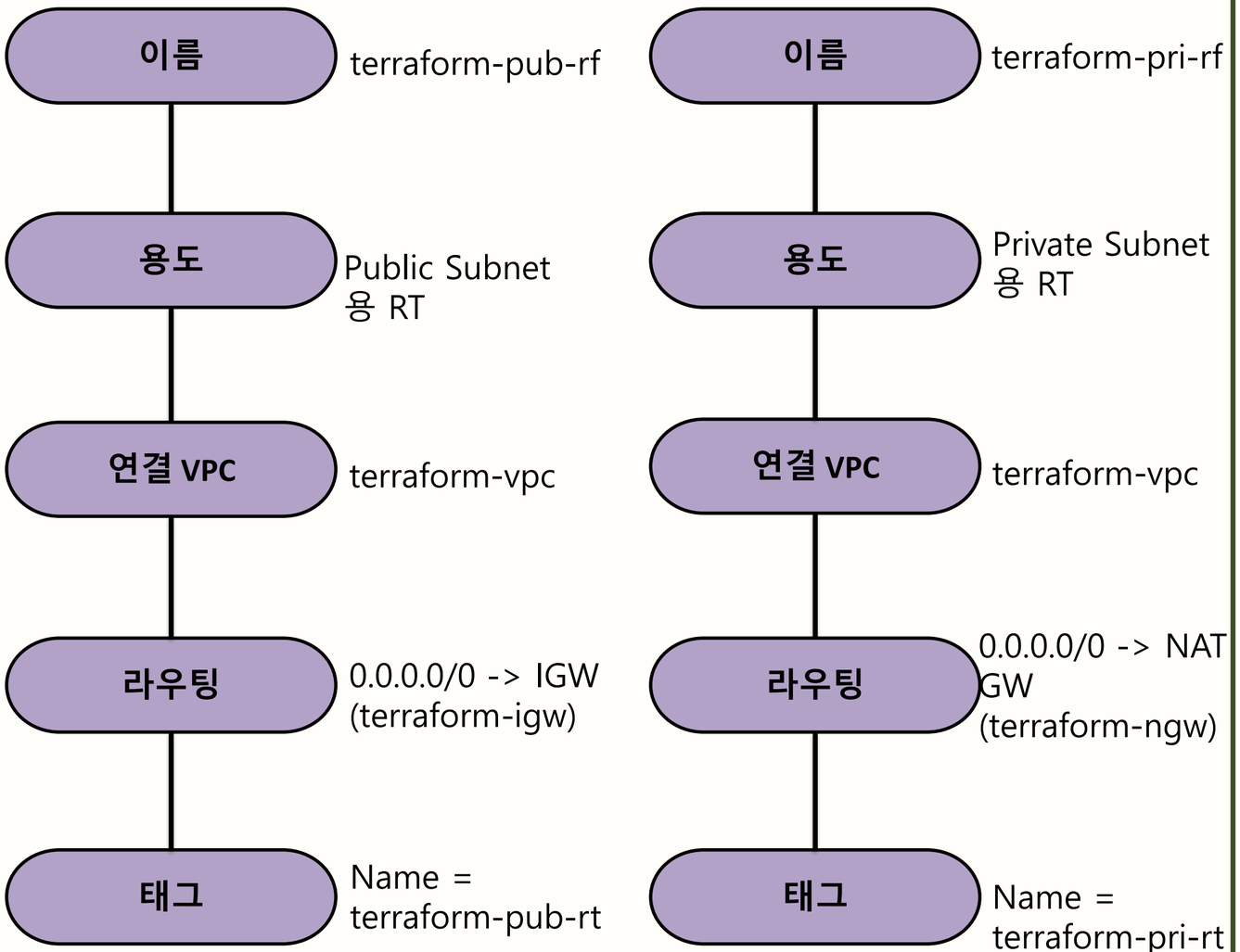
```
# 라우팅 테이블
resource "aws_route_table" "terraform-pub-rt" {
  vpc_id = aws_vpc.terraform-vpc.id

  tags = {
    "Name" = "terraform-pub-rt"
  }
}

resource "aws_route_table" "terraform-pri-rt" {
  vpc_id = aws_vpc.terraform-vpc.id

  tags = {
    "Name" = "terraform-pri-rt"
  }
}

...
```



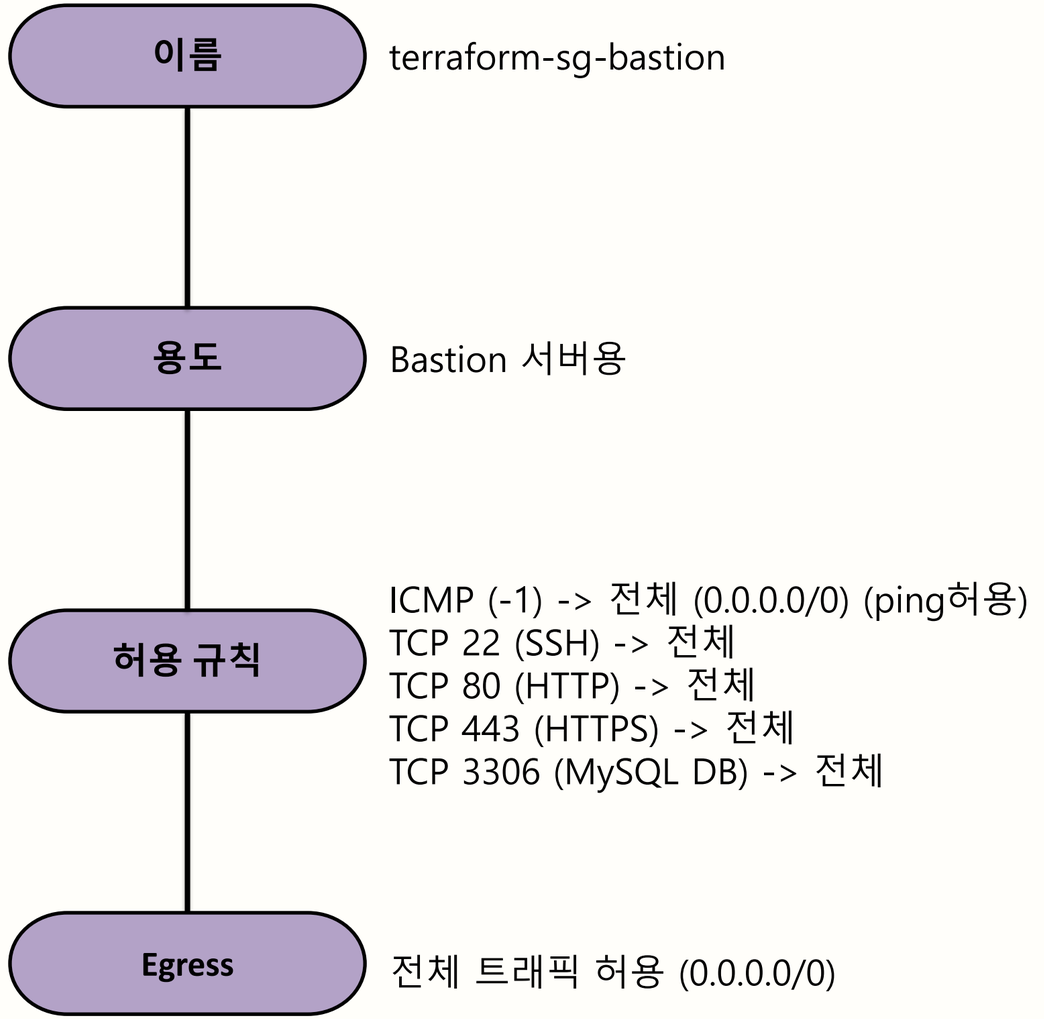
# 7. Security Group(Bastion SG)



```
# Bastion SG
resource "aws_security_group" "terraform-sg-
bastion" {
  name        = "terraform-sg-bastion"
  description = "for Bastion Server"
  vpc_id      = aws_vpc.terraform-vpc.id
  tags = {
    "Name" = "terraform-sg-bastion"
  }

  ingress {
    from_port   = -1
    to_port     = -1
    protocol    = "icmp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "for ping"
  }

...
```



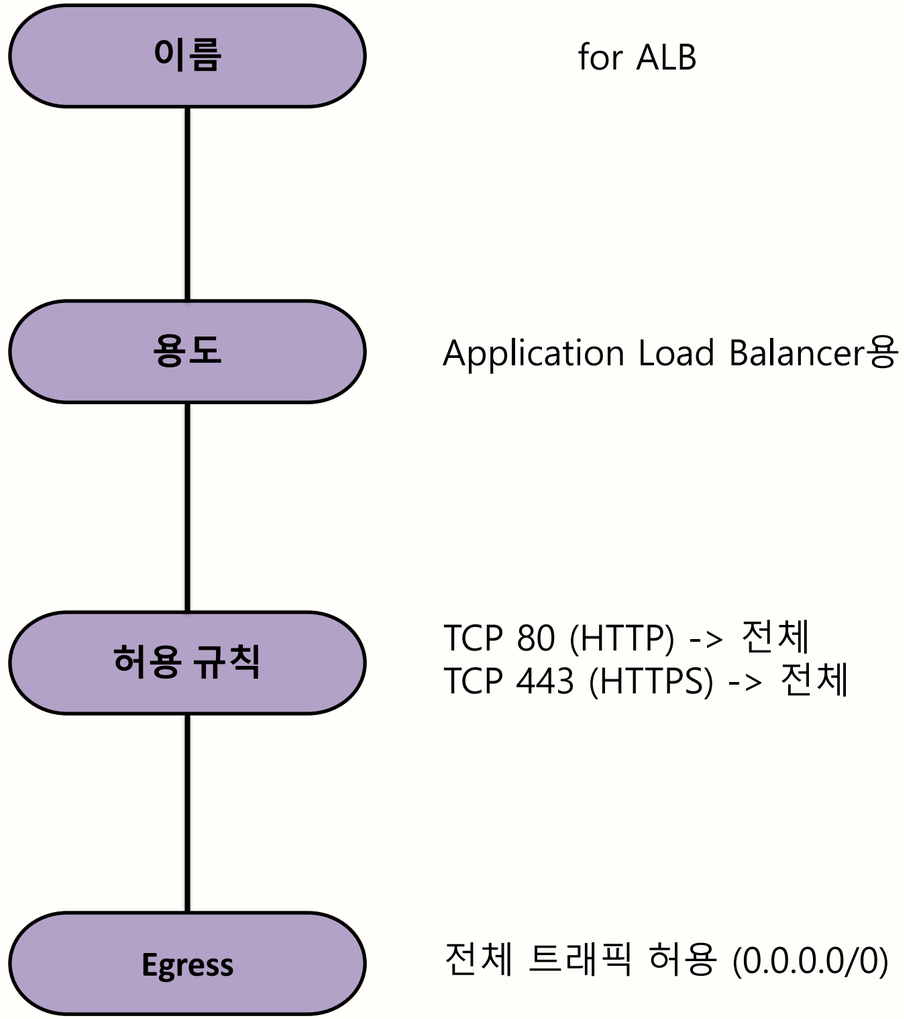
# 7. Security Group(ALB SG)



```
# ALB SG
resource "aws_security_group" "terraform-sg-
alb" {
  name    = "for ALB"
  vpc_id = aws_vpc.terraform-vpc.id

  ingress {
    from_port    = 80
    to_port      = 80
    protocol     = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "for HTTP"
  }

...
```



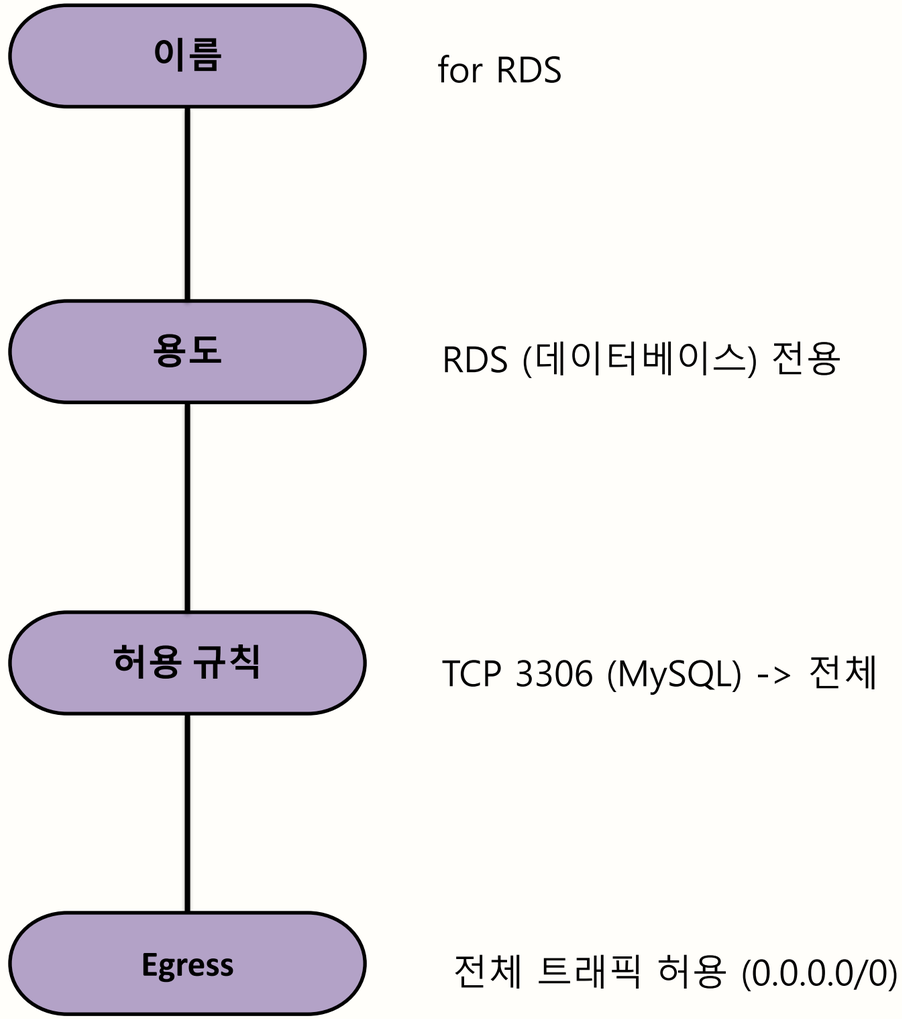
# 7. Security Group(RDS SG)



```
# RDS SG
resource "aws_security_group" "terraform-sg-rds" {
  name   = "for RDS"
  vpc_id = aws_vpc.terraform-vpc.id

  ingress {
    from_port   = 3306
    to_port     = 3306
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "for DB"
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

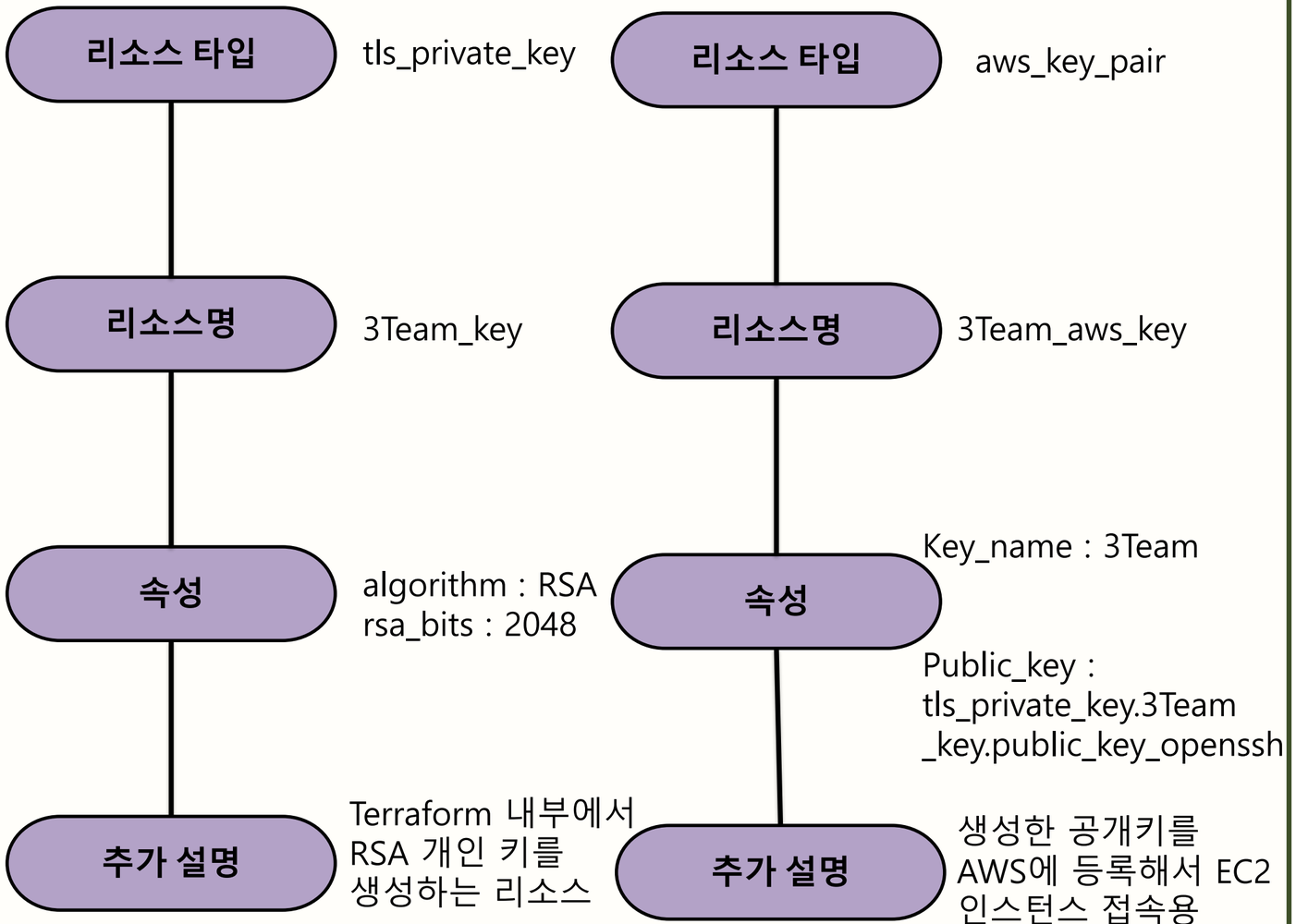


# 8. Key_Pair

# 5. TF 파일 분석

```
#key_Pair
resource "tls_private_key" “3Team_key" {
  algorithm = "RSA"
  rsa_bits   = 2048
}

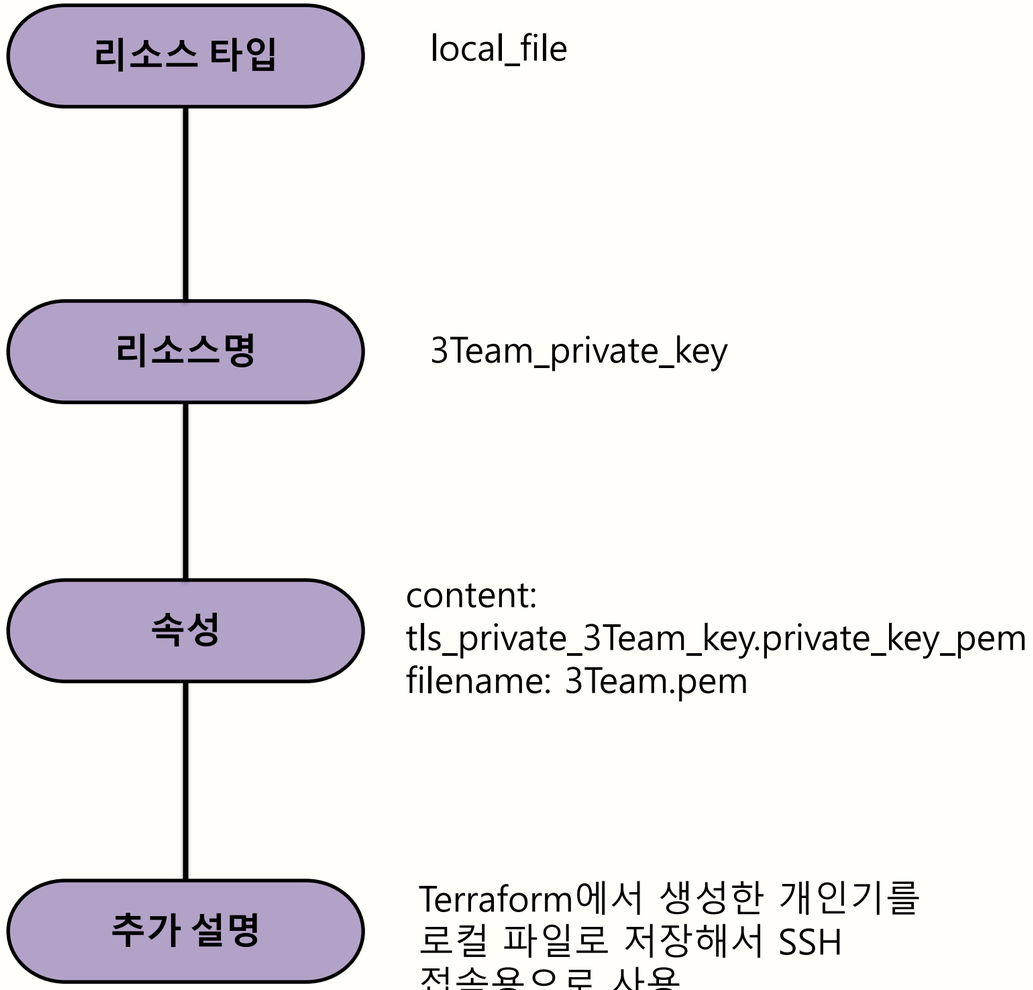
resource "aws_key_pair" “3Team_aws_key" {
  key_name    = “3Team"
  public_key =
tls_private_key.3Team_key.public_key_openssh
}
```



# 8. Key_Pair (local_file)



```
resource "local_file" “3Team_private_key" {
  content  =
tls_private_key.3Team_key.private_key_pem
  filename = “3Team.pem"
}
```

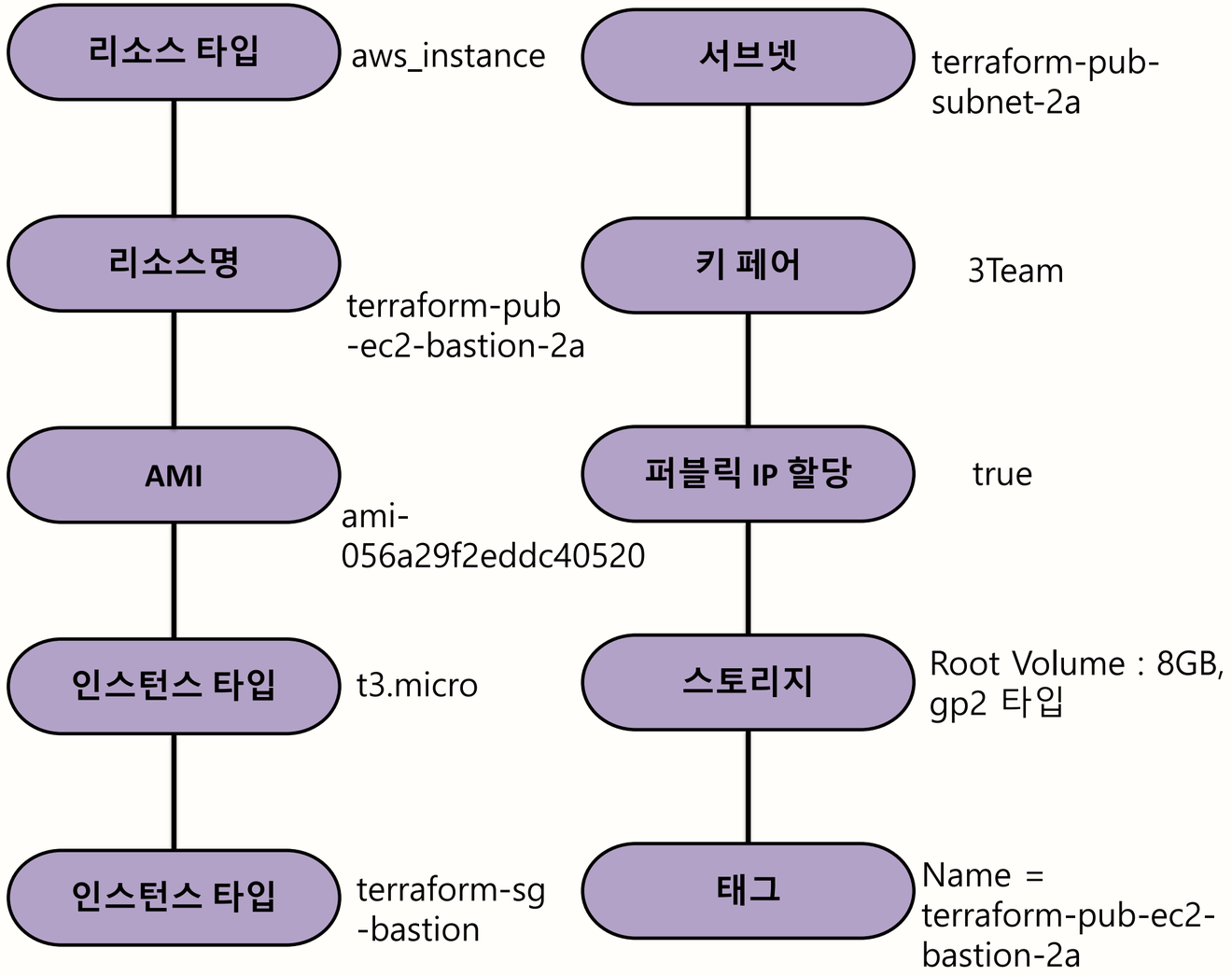


# 9. EC2

# 5. TF 파일 분석

```
resource "aws_instance" "terraform-pub-ec2-bastion-2a" {
  ami                         = "ami-056a29f2eddc40520"
  instance_type               = "t3.micro"
  vpc_security_group_ids      = [aws_security_group.terraform-sg-bastion.id]
  subnet_id                   = aws_subnet.terraform-pub-subnet-2a.id
  key_name                    = “3Team"
  associate_public_ip_address = true

  root_block_device {
    volume_size = "8"
    volume_type = "gp2"
    tags = {
      "Name" = "terraform-pub-ec2-bastion-2a"
    }
  }
…
```

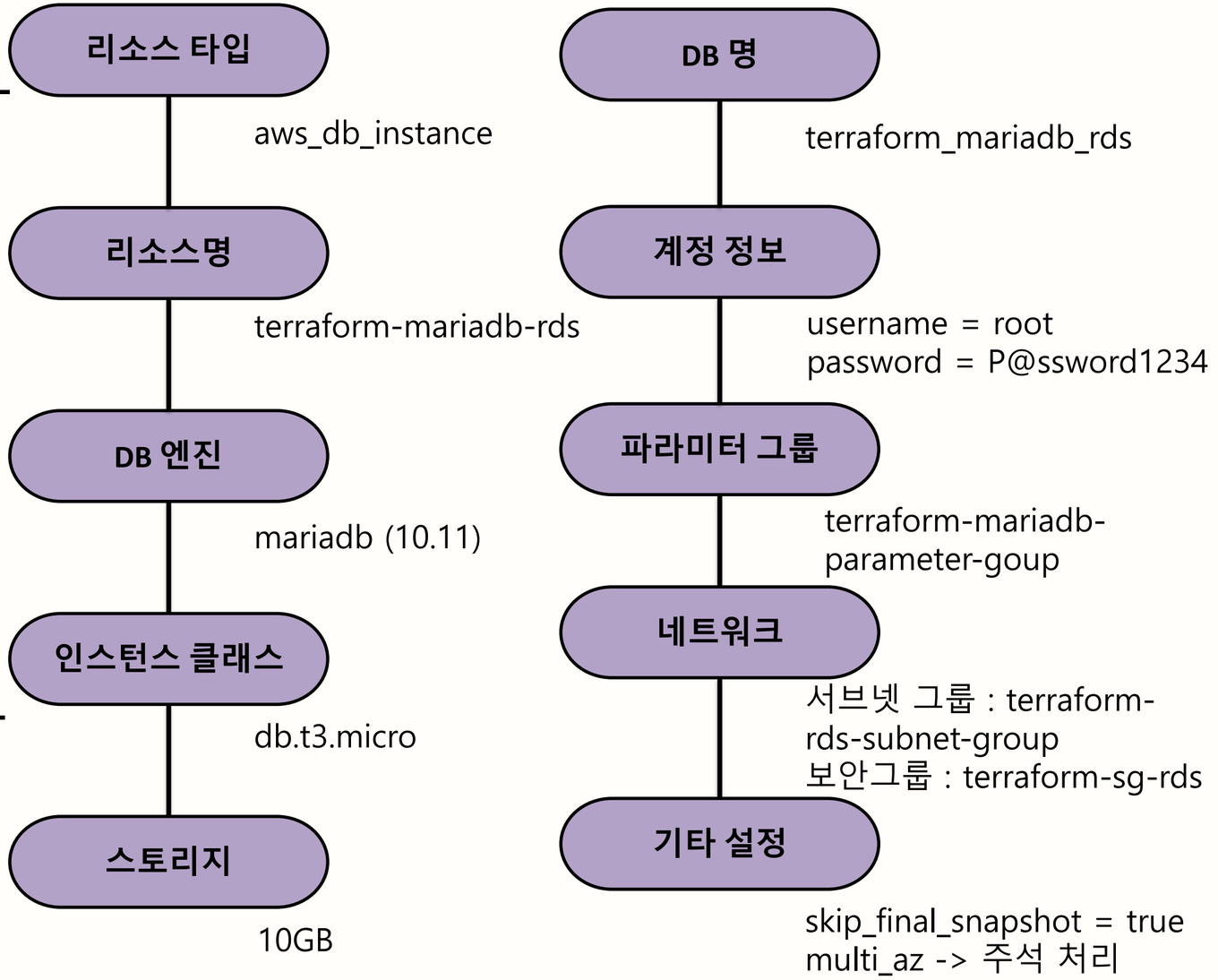


# 10. RDS (RDS instance {MariaDB})



```
# RDS 생성
resource "aws_db_instance" "terraform-mariadb-
rds" {
  identifier_prefix     = "terraform-mariadb-rds"
  allocated_storage     = 10
  engine                = "mariadb"
  engine_version        = "10.11"
  instance_class        = "db.t3.micro"
  db_name               = "terraform_mariadb_rds“
# 영문자로 시작해야 하며 영문자와 숫자, _만 가능
  username              = "root"
  password              = "P@ssword1234"
# 8자 이상, 영문 대소문자, 숫자 및 특수 문자를
혼합하여 사용
  parameter_group_name = "terraform-mariadb-
parameter-group"
  skip_final_snapshot   = true


…
```



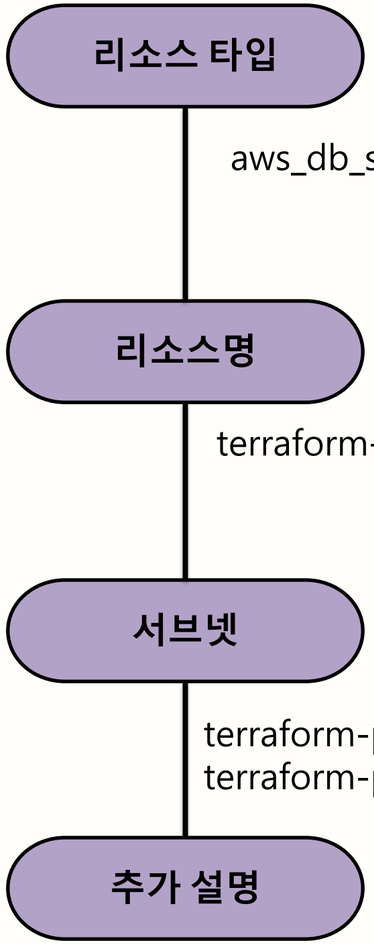
# 10. RDS



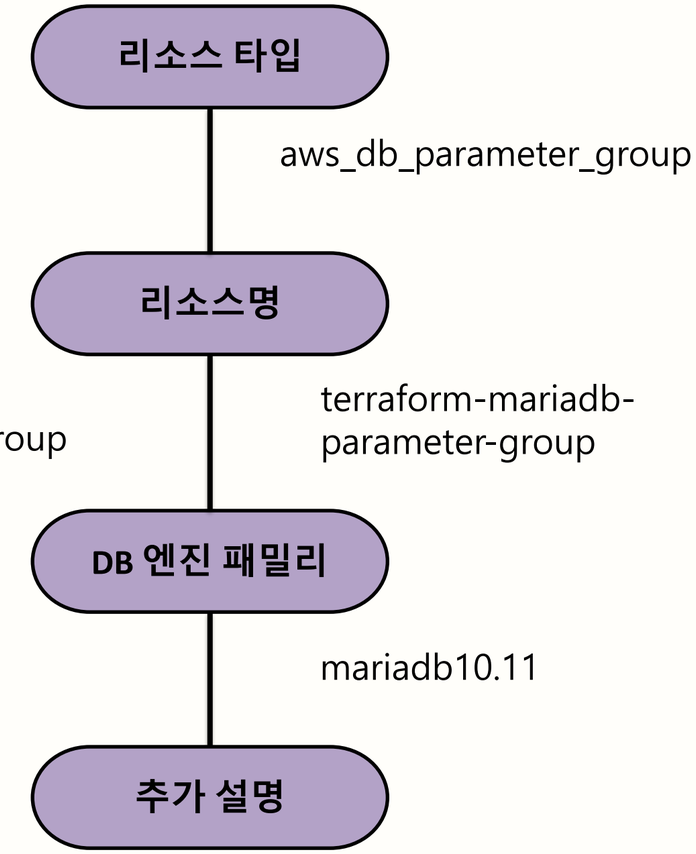
```
# RDS 서브넷 그룹 생성
resource "aws_db_subnet_group" "terraform-rds-subnet-group" {
  name       = "terraform-rds-subnet-group"
  subnet_ids = [aws_subnet.terraform-pri-subnet-2a.id, aws_subnet.terraform-pri-subnet-2c.id]

  tags = {
    Name = "terraform-rds-subnet-group"
  }
}

resource "aws_db_parameter_group" "terraform-mariadb-parameter-group" {
  name       = "terraform-mariadb-parameter-group"
  family     = "mariadb10.11"
# 사용 중인 MariaDB 버전에 맞게 조정
  description = "Custom parameter group for MariaDB"
```




RDS 인스턴스를 퍼블릭이 아닌 프라이빗 서브넷에 배치하여 보안 강화 ,다중 AZ 배포 시 여러 가용 영역 지원

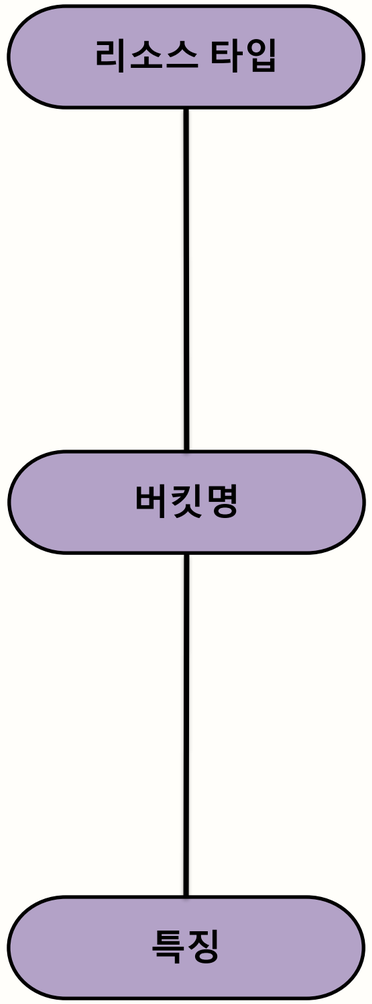



max_connections = 150
time_zone = Asia/Seoul
character-set_* =utf8mb4
collation_* = utfmb4_Unicode_ci
binlog_format = ROW

# 11. S3

# 5. TF 파일 분석

```
resource “3Team_s3_bucket" “3Team-demo" {
  bucket = “3Team-demo"
}
```




3Team_s3_bucket

3Team-demo

버킷명은 글로벌 유일이기
때문에 팀명인 3Team으로 지정

# 12. acm

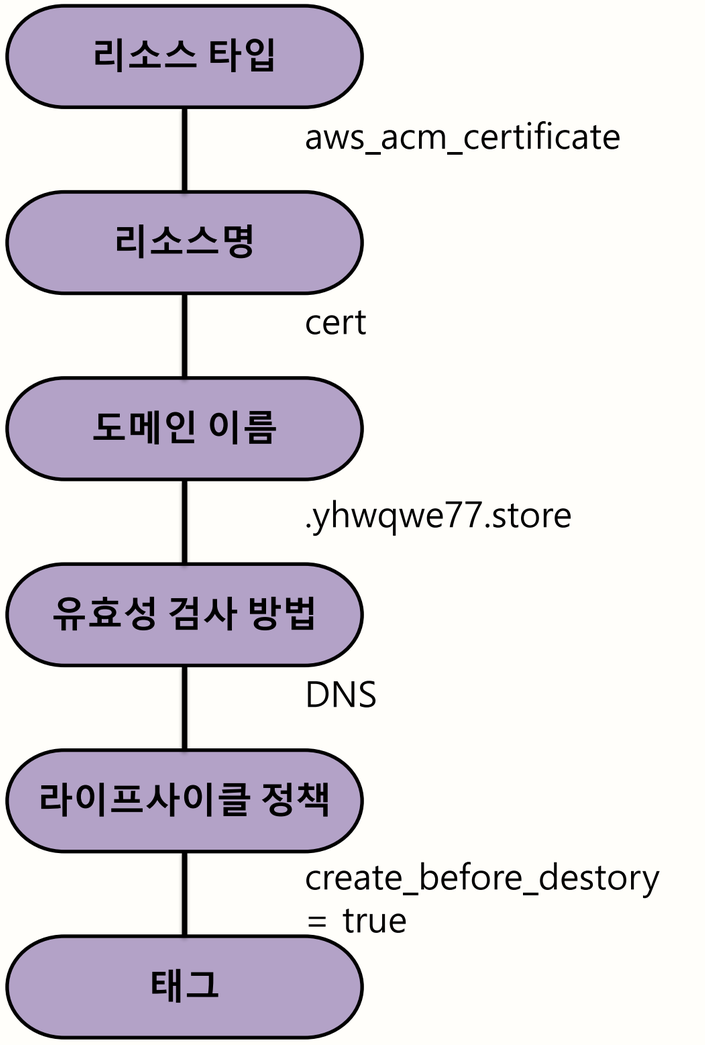
# 5. TF 파일 분석

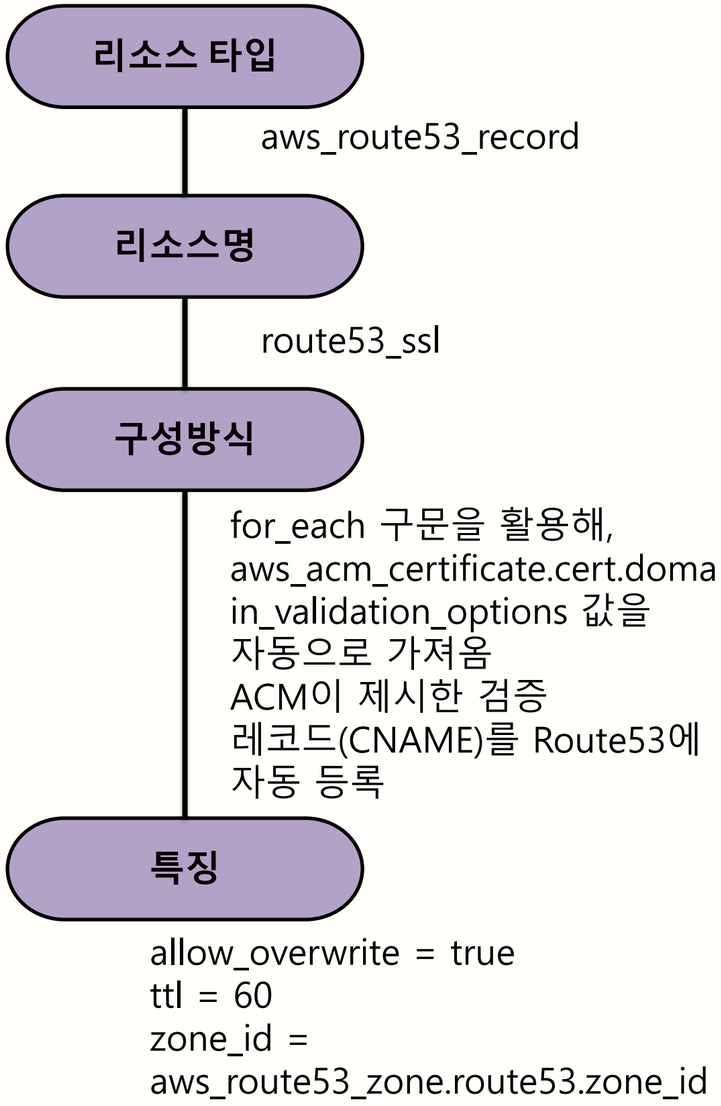
```
# 인증서 생성
resource "aws_acm_certificate" "cert" {
  domain_name       = "*.yhwqwe77.store"
  validation_method = "DNS"

  tags = {
    Name = "yhwqwe77.store"
  }

  lifecycle {
    create_before_destroy = true
  }
}

...
```

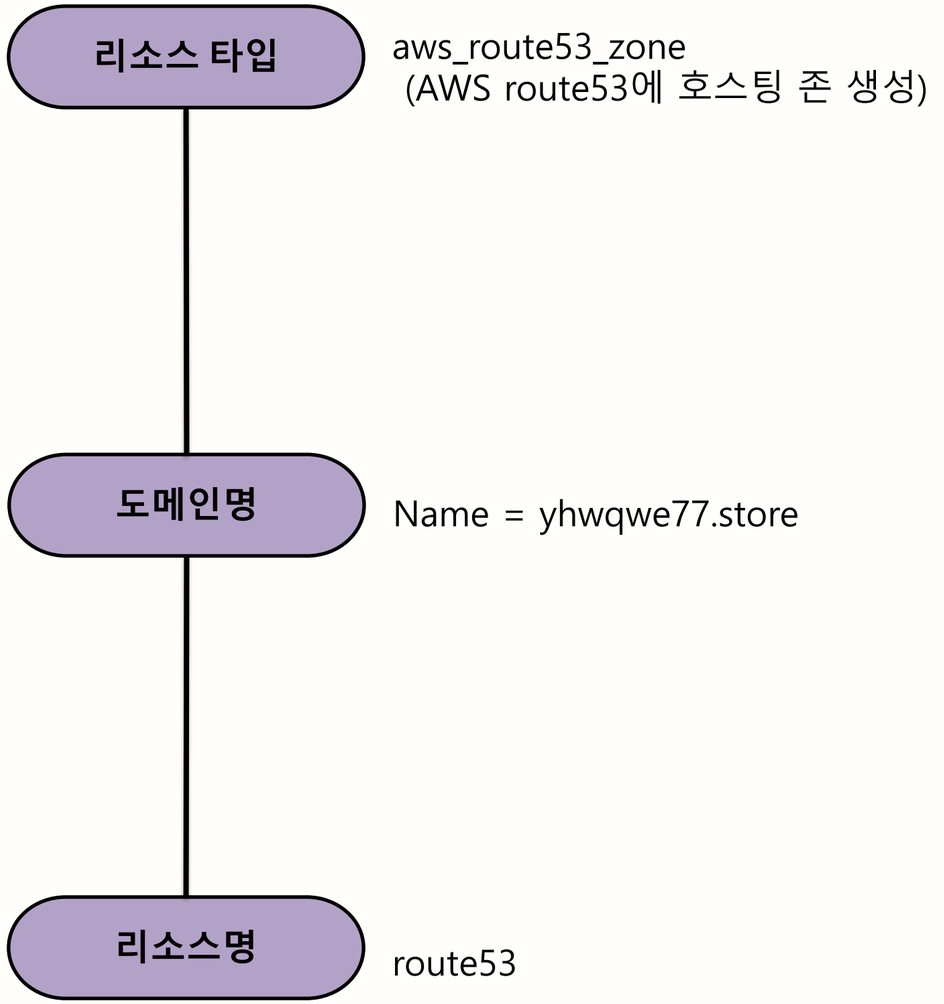


# 13. Route53



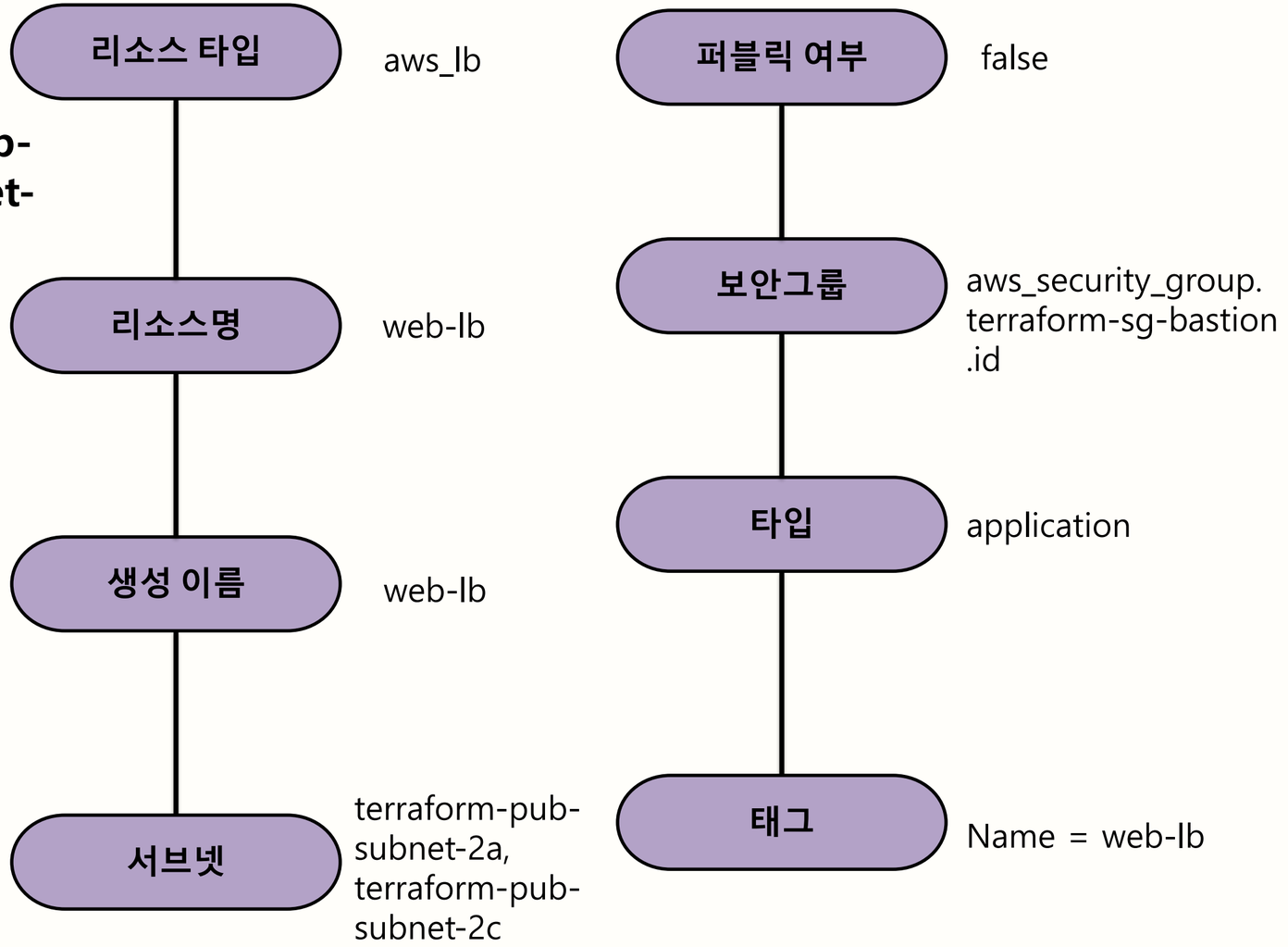
```
# Route53
resource "aws_route53_zone" "route53" {
  name = "yhwqwe77.store"
}
```



# 14. Alb (Application Load Balancer)

# 5. TF 파일 분석

```
resource "aws_lb" "web-lb" {
  name               = "web-lb"
  subnets            = [aws_subnet.terraform-pub-subnet-2a.id, aws_subnet.terraform-pub-subnet-2c.id]
  internal           = false
  security_groups    = [aws_security_group.terraform-sg-bastion.id]
  load_balancer_type = "application"
  tags = {
    "Name" = "web-lb"
  }
}
```



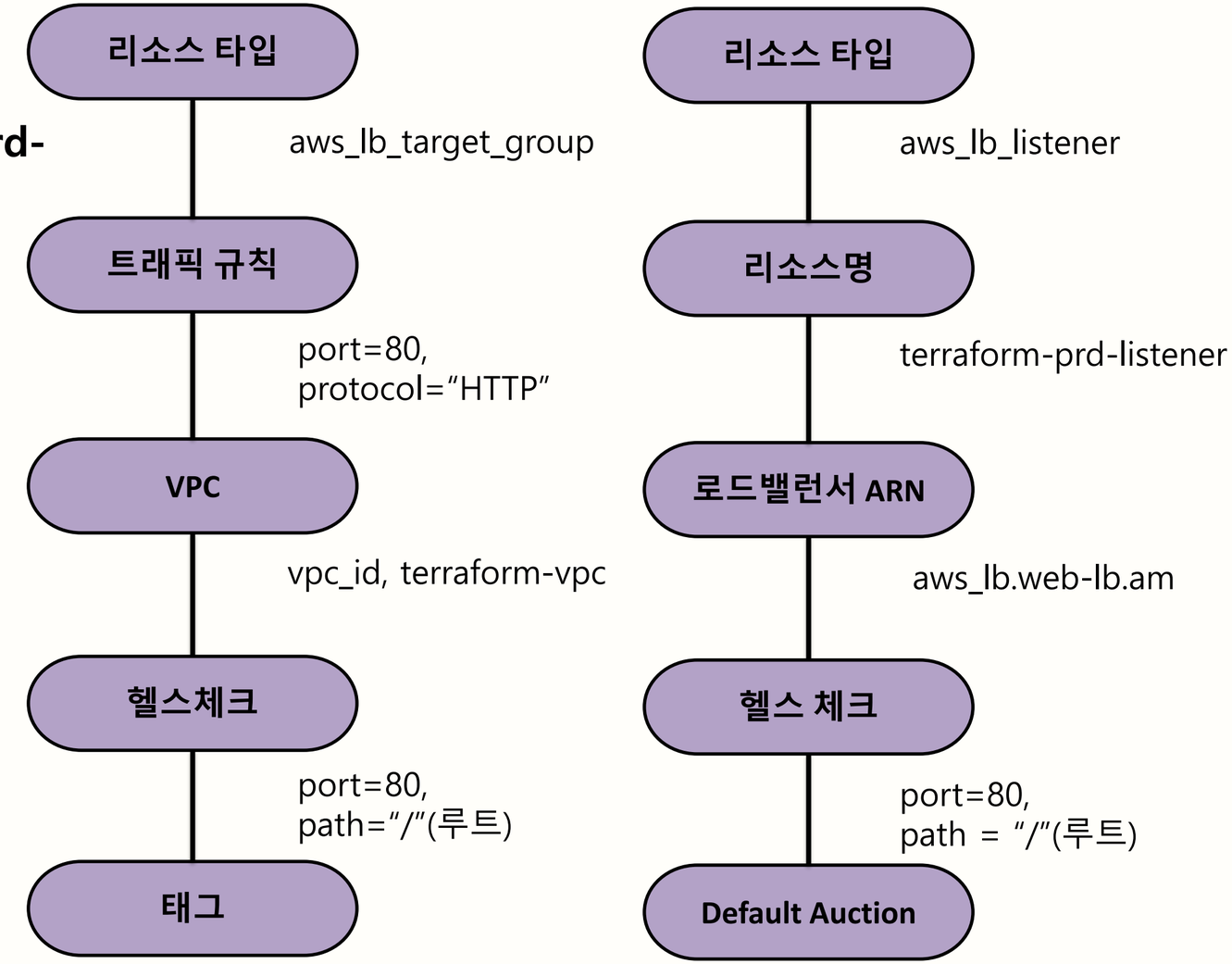
# 14. Alb

# 5. TF 파일 분석

```
############## Target Group

resource "aws_lb_target_group" "terraform-prd-
tg" {
  name     = "terraform-prd-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.terraform-vpc.id
  health_check {
    port = 80
    path = "/"
  }
  tags = {
    "Name" = "terraform-prd-tg"
  }
}

...
```



# 15. Auto Scaling (Launch Configuration)

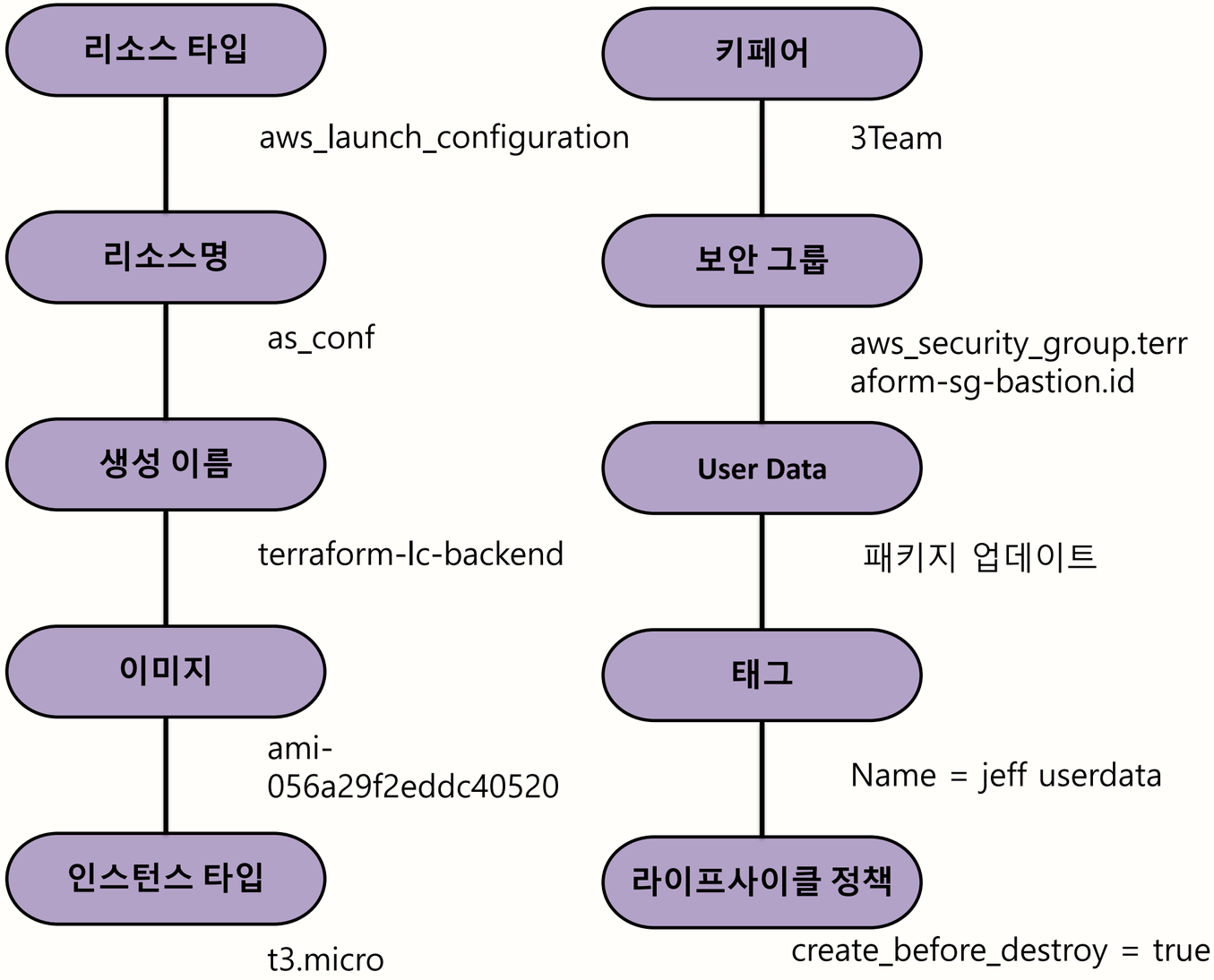
# 5. TF 파일 분석

```
#AutoScailing
resource "aws_launch_template" "as_conf" {
  name_prefix    = "terraform-lc-backend-"
  image_id       = "ami-056a29f2eddc40520"
  instance_type = "t3.micro"
  key_name       = “3Team"

 vpc_security_group_ids =
[aws_security_group.terraform-sg-bastion.id]

  user_data = base64encode(<<-EOF
    #!/bin/bash
    sudo apt update
    sudo apt install -y nginx
  EOF

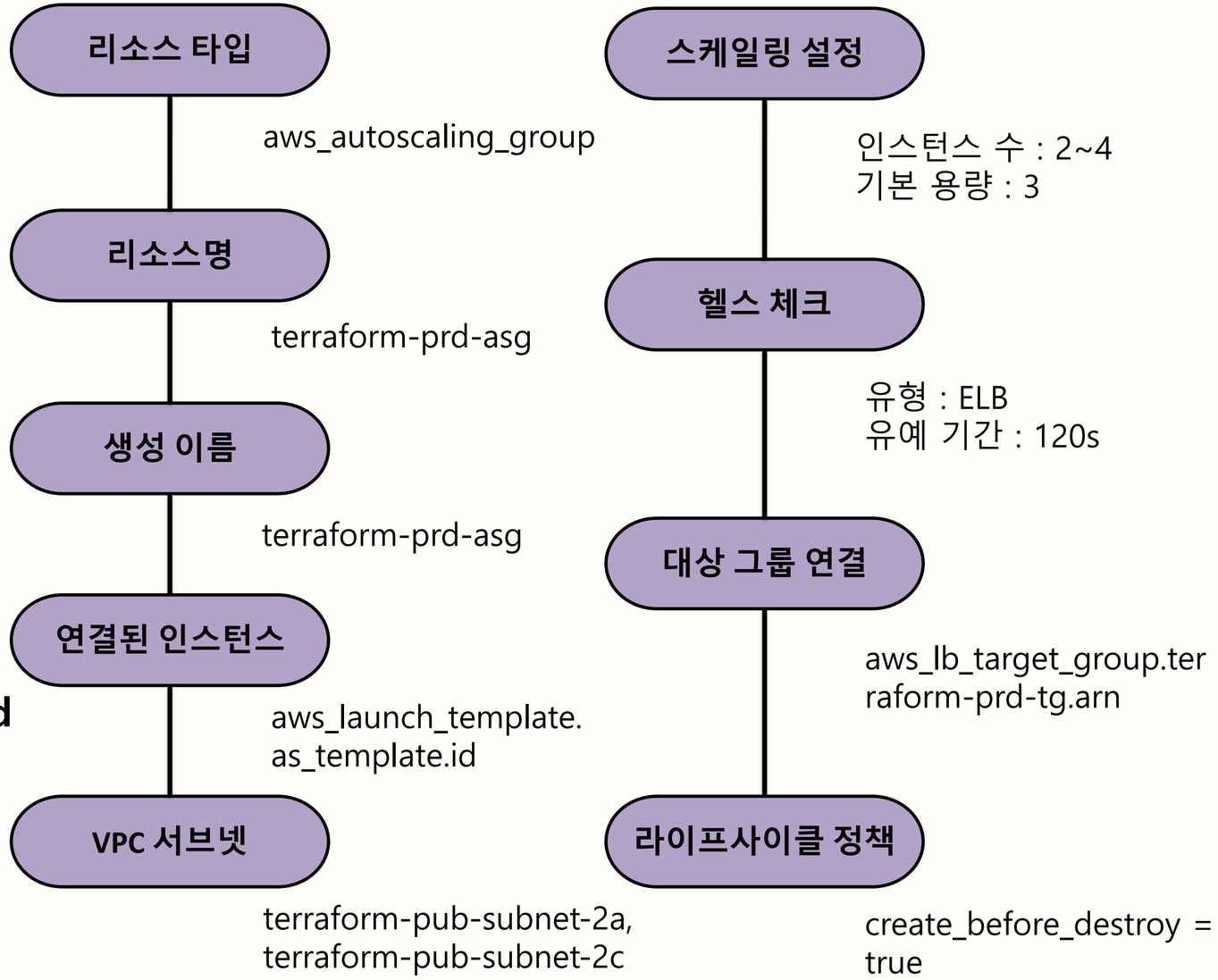
…
```



# 15. Auto Scaling (Auto Scaling Group)

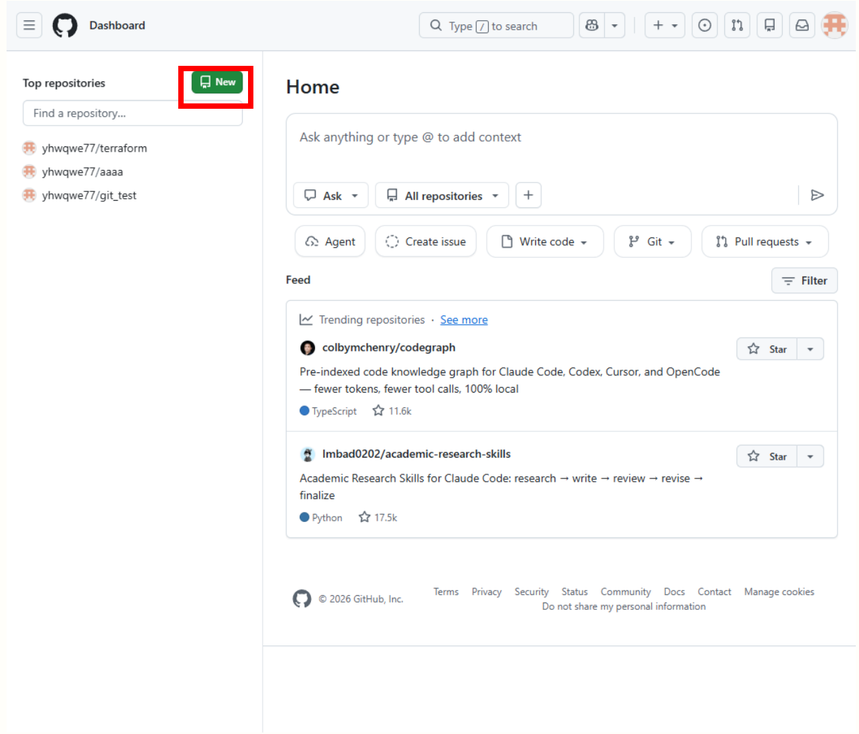
# 5. TF 파일 분석

```
resource "aws_autoscaling_group" "terraform-
prd-asg" {
  name                      = "terraform-prd-asg"
  vpc_zone_identifier       =
[aws_subnet.terraform-pub-subnet-2a.id,
aws_subnet.terraform-pub-subnet-2c.id]
  min_size                  = 2
  max_size                  = 4
  desired_capacity          = 3
  health_check_grace_period = 120
  health_check_type         = "ELB"
  target_group_arns         =
[aws_lb_target_group.terraform-prd-tg.arn]
  launch_template {
    id      = aws_launch_template.as_template.id
    version = "$Latest"    }
  lifecycle {
    create_before_destroy = true
  }
}
```

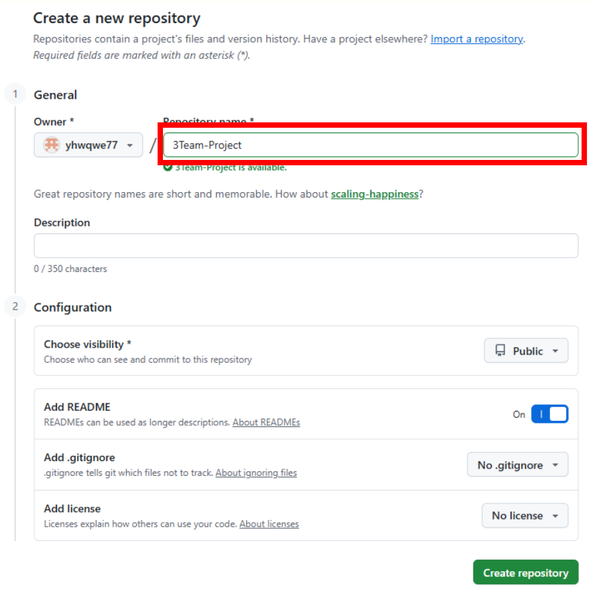


# 6. GitHub, Terraform Cloud, AWS 연동

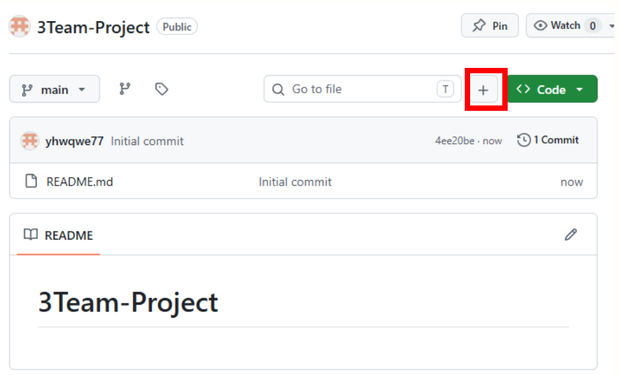
## 6-1. GitHub Repository 만들기




Github Repository 만들기



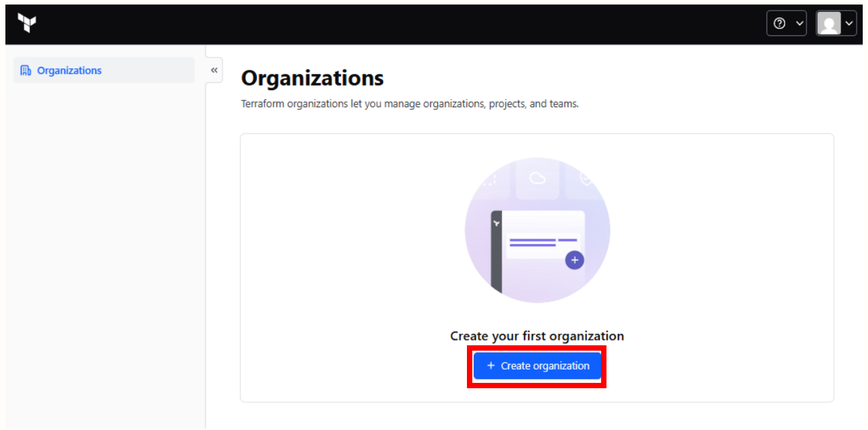

Repository 생성



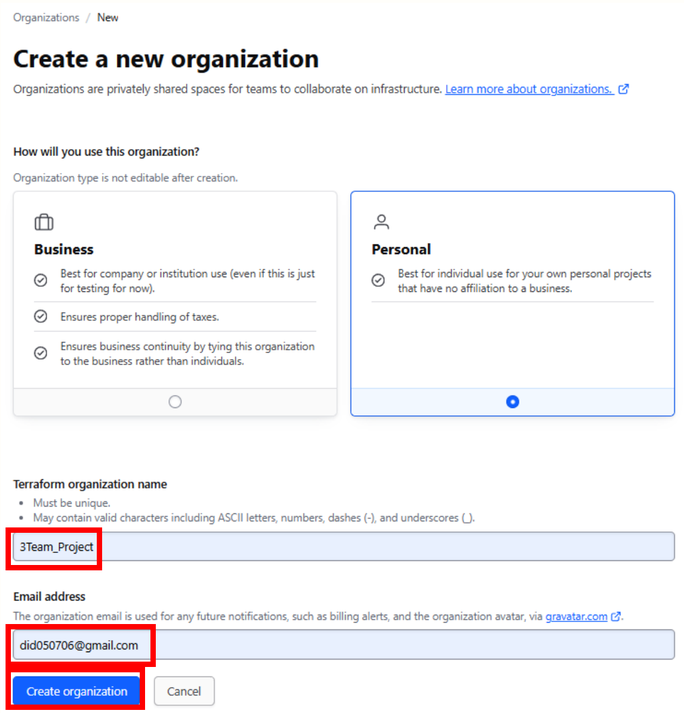

Repository 이름 설정
README 파일 추가

# 6. GitHub, Terraform Cloud, AWS 연동

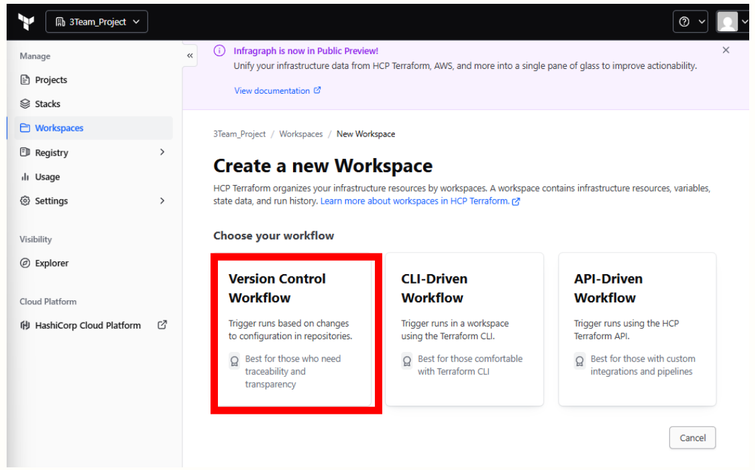
## 6-2. Terraform Cloud 초기 구성 & GitHub 연동




1. Organizations 생성



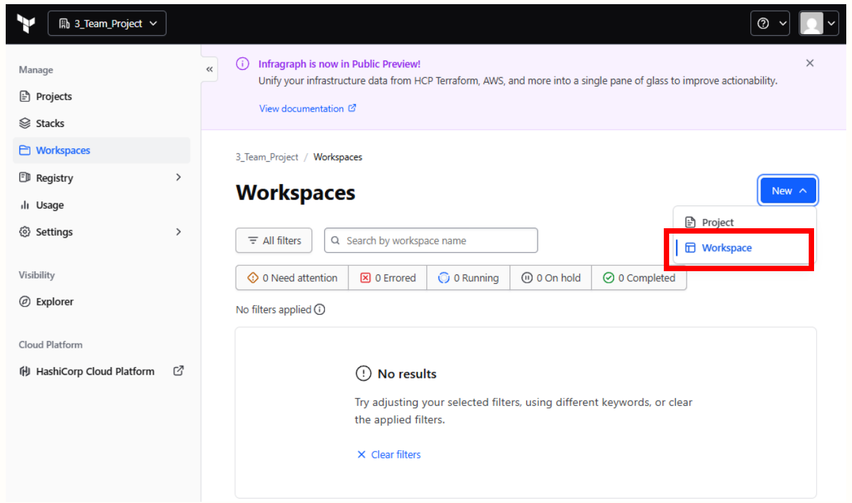

2.
organizaions이름&이메일설정



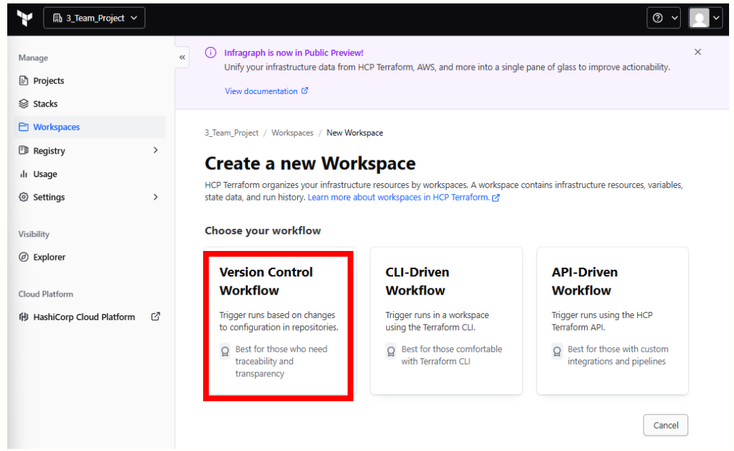

3. workspace 생성 및 version control workflow 선택

# 6. GitHub, Terraform Cloud, AWS 연동

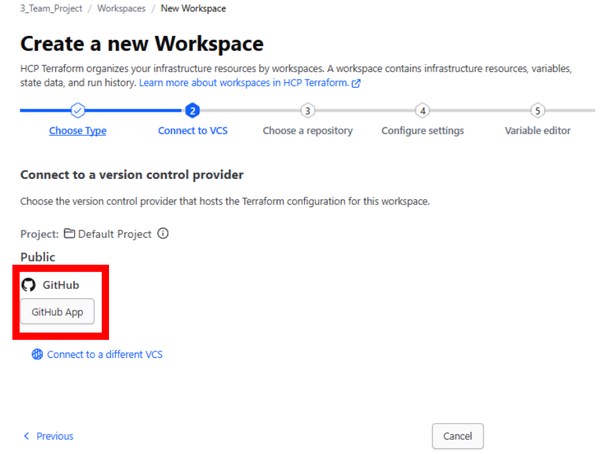
## 6-2. Terraform Cloud 초기 구성 & GitHub 연동




4. workspace 생성



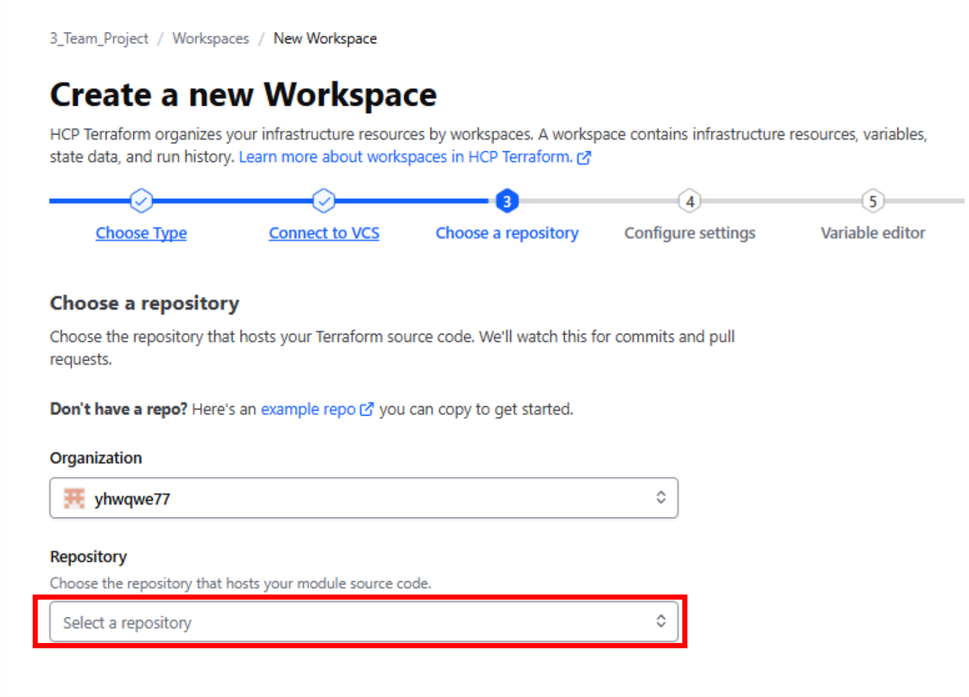

4-1.version control workflow 선택



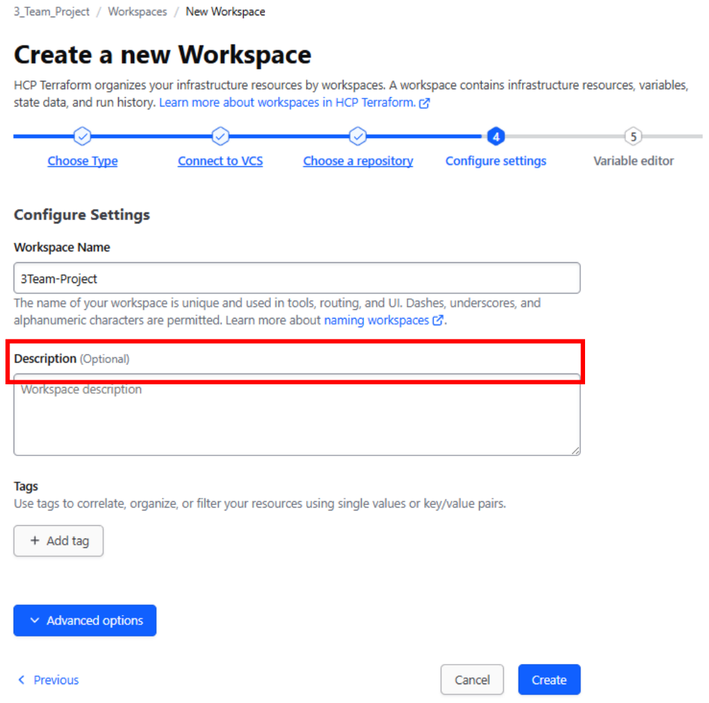

4-2. github 선택

# 6. GitHub, Terraform Cloud, AWS 연동

## 6-2. Terraform Cloud 초기 구성 & GitHub 연동



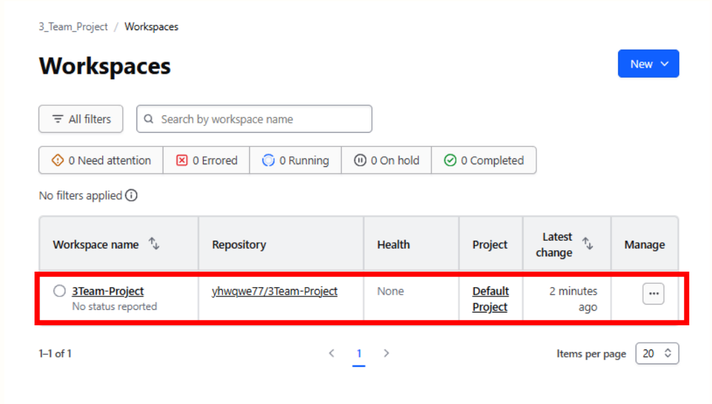

4-3. repository 선택



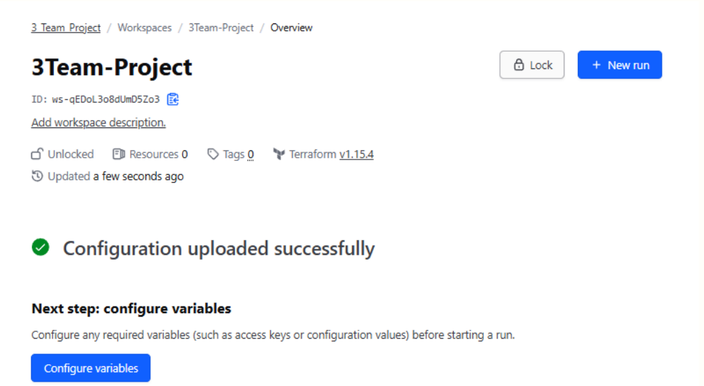

4-4. configure settings

# 6. GitHub, Terraform Cloud, AWS 연동

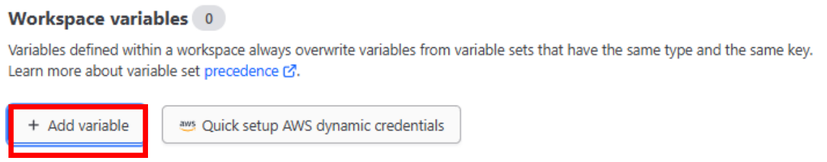
## 6-3. Terraform Cloud & AWS 연동




1. workspace 선택



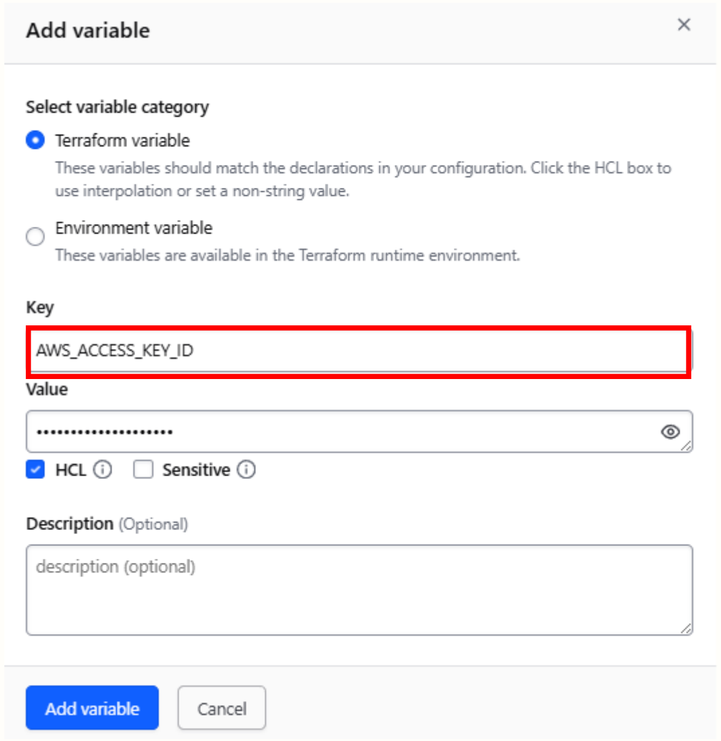

2. configure variables



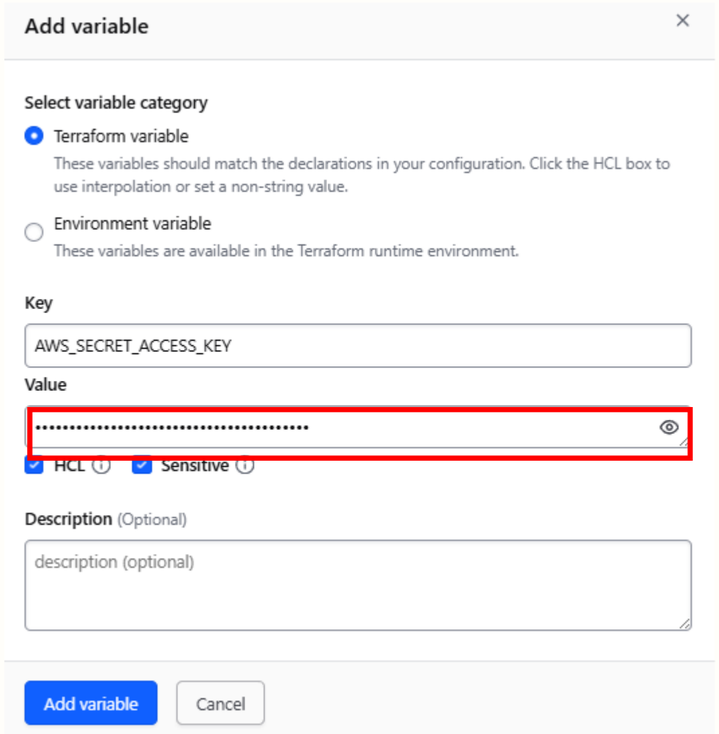

3. add variable

# 6. GitHub, Terraform Cloud, AWS 연동

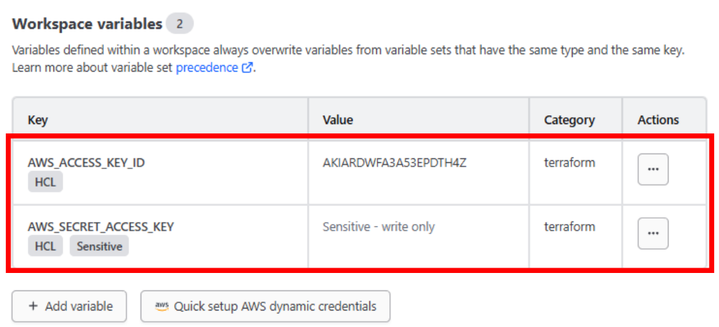
## 6-3. Terraform Cloud & AWS 연동




4. access key id



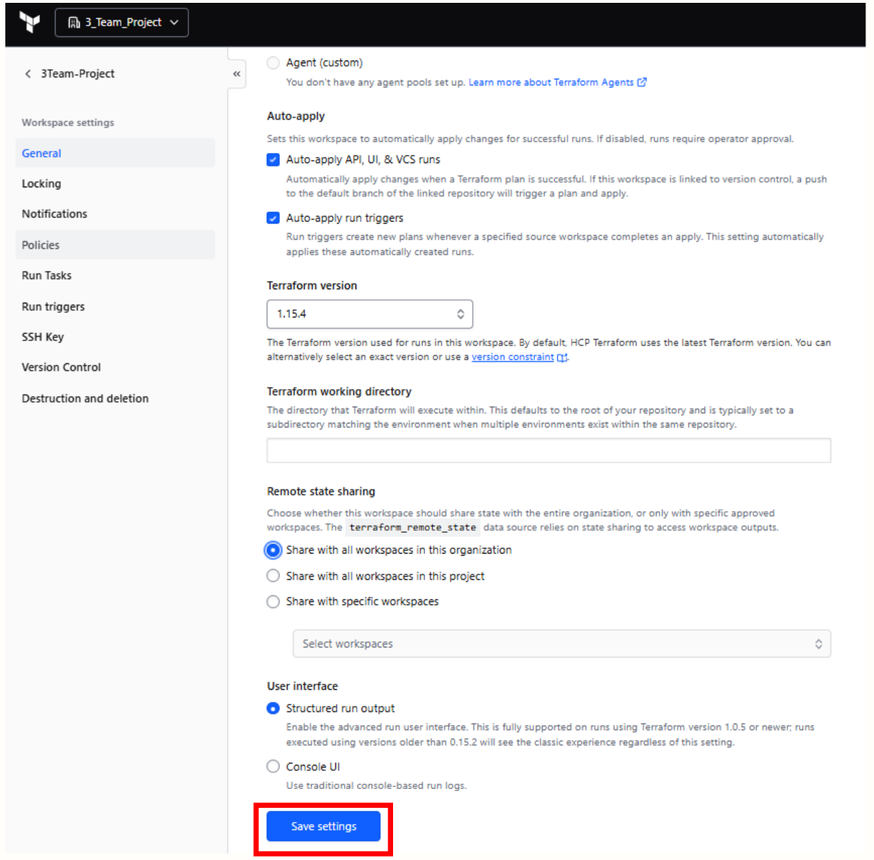

5. secret access key




6. 키 등록 확인

# 6. GitHub, Terraform Cloud, AWS 연동

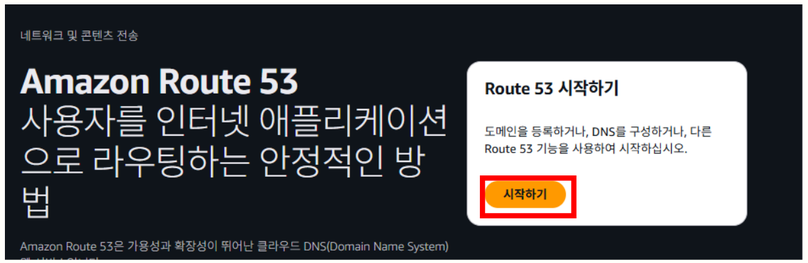
## 6-3. Terraform Cloud & AWS 연동




7. general setting

# 7. Route53 도메인 등록

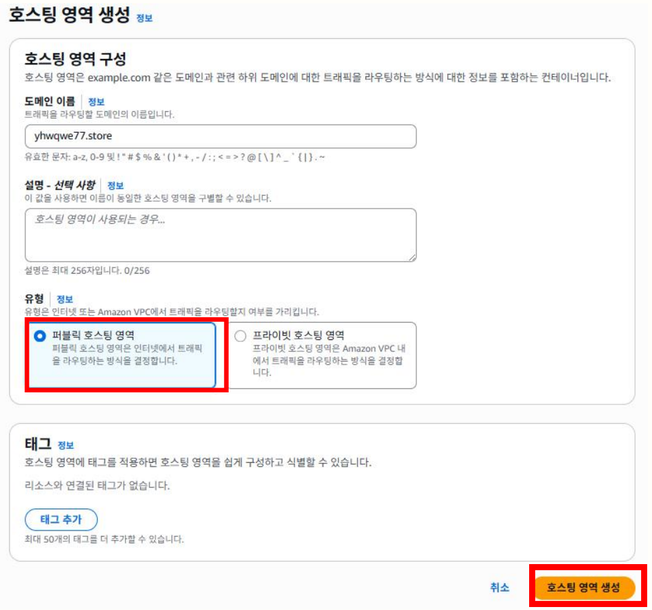
## 7-1. 호스트 영역 생성




1. route53시작



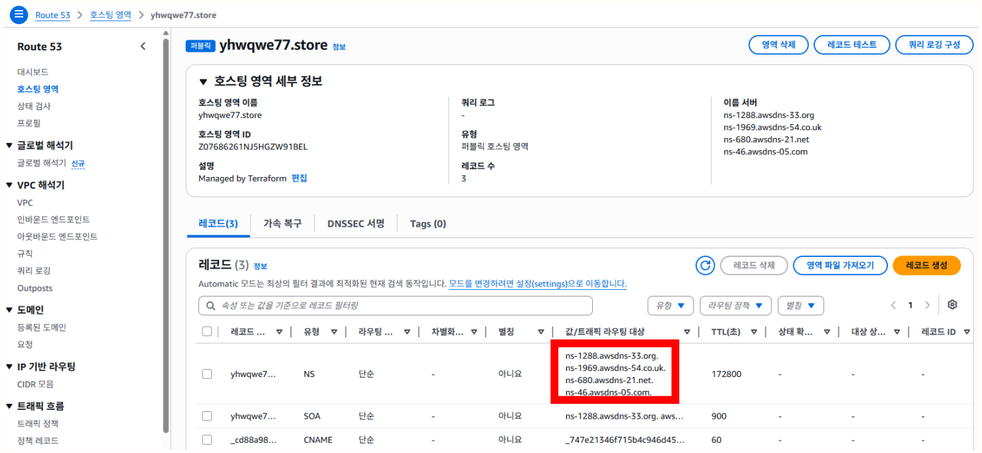

2.시작지점
선택



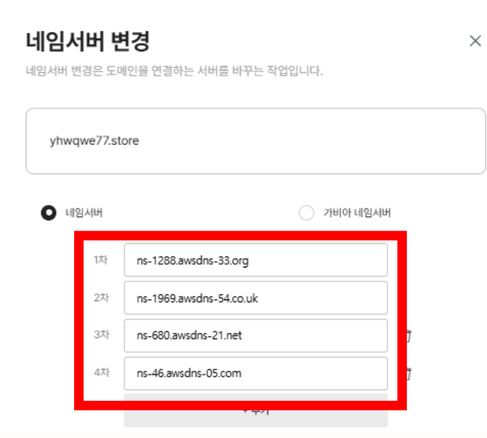

3.호스트 영역 이름 및 연결유형 설정

# 7. Route53 도메인 등록

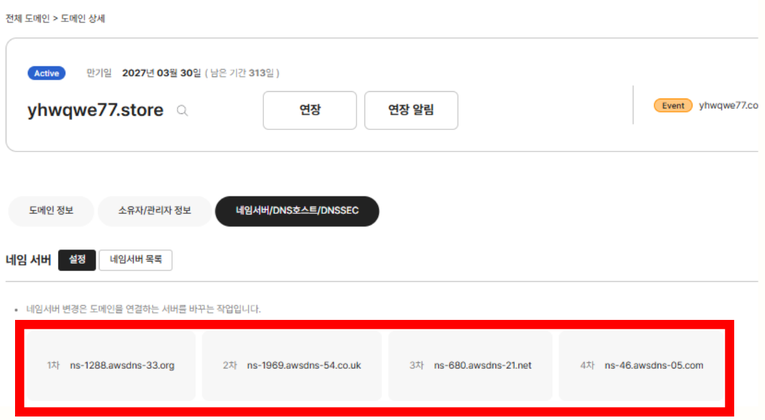
## 7-2. 도메인 등록업체 NS 레코드 입력




1. 네임서버확인



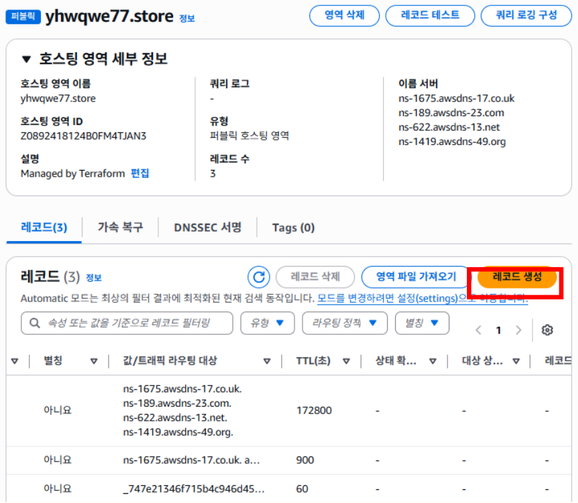

2. 가비아 네임서버 변경



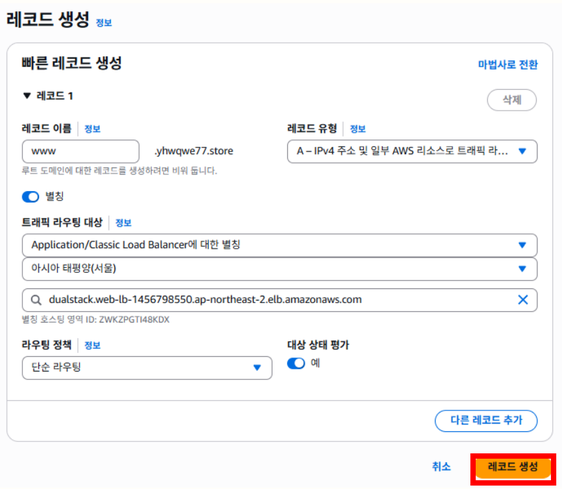

3. 네임서버 변경 확인

# 7. Route53 도메인 등록

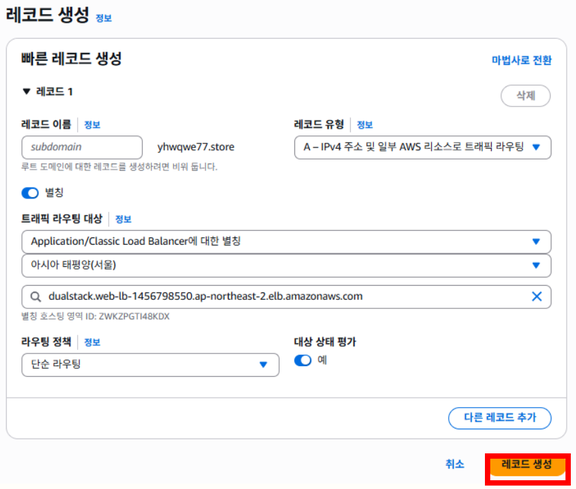
## 7-3. 가비아에서 확인



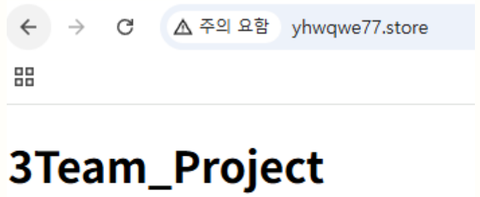

1.레코드 생성




2.레코드1 생성




3.레코드2 생성(www)




4. 도메인 작동 확인

# Terraform Project

감사합니다.
백우진 서승덕 양혜원 한준영